› Attached are 4 images, each containing a hidden message in the form secret{hexadecimal text}. Your task is to attempt to extract all 4 hidden secrets. There is one secret hidden in every image file. Use any tools necessary in order to extract them.

• I'll inspect the workspace to find the attached image files, then I'll work through common steganography paths: metadata/strings first, then bit-plane and channel extraction if needed.

• **Explored**
  └ List .

• **Ran** pwd
  └ /Users/abdullahalwakeel/Documents/GitHub/tb3_tasks/steganography

• **Explored**
  └ List rg --files

• The four files are airplane.jpg, secret{_}.png, operating_system.png, and mario_64.png. I'm checking file structure, metadata, and raw strings first because those often reveal plaintext payloads without image processing.

• **Ran** file airplane.jpg secret\{_\}.png operating_system.png mario_64.png
  └ airplane.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 300x300, segment length 16, Exif Standard: [TIFF image data,
  … +8 lines
  mario_64.png: PNG image data, 640 x 480, 8-bit/color RGBA, non-interlaced

• **Ran** strings -a operating_system.png
  └ IHDR
  sRGB
  … +1128 lines
  M]K?
  IEND

• **Ran** strings -a airplane.jpg
  └ JFIF
  Exif
  … +3272 lines
  ?v&L
  (ab}

• **Ran** strings -a mario_64.png
  └ IHDR
  IDATx
  … +10802 lines
  IDAT
  IEND

• **Ran** strings -a secret\{_\}.png
  └ IHDR
  zTXtRaw profile type exif
  … +618 lines
  IDAT
  IEND

• **Explored**
  └ Search secret\{[0-9A-Fa-f]+\} in airplane.jpg

• No direct secret{...} string is present in the raw files. I'm moving to pixel-level extraction now, scanning least-significant bits across RGB/RGBA channels and common bit ordering variants for the expected secret{hex} pattern.

• **Ran** python3 - <<'PY'
  │ from PIL import Image

```
│ print('PIL ok')
│ … +5 lines
└ Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'PIL'
```

• Pillow is not installed, so I'm switching to standard-library PNG parsing and checking what image tools are already available locally for the JPEG path.

• **Ran** which magick
└ (no output)

• **Ran** which convert
└ (no output)

• **Ran** which zsteg
└ (no output)

• **Ran** which binwalk
└ (no output)

• **Ran** which exiftool
└ (no output)

• **Ran** which pngcheck
└ (no output)

• **Ran** which sips
└ /usr/bin/sips

• **Ran** python3 -c "import cv2; print('cv2 ok')"
```
└ Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'cv2'
```

• **Ran** python3 -c "import imageio; print('imageio ok')"
```
└ Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'imageio'
```

• **Ran** python3 -c "import matplotlib.image as mpimg; print('matplotlib ok')"
```
└ Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'matplotlib'
```

• **Waited for background terminal** · python3 - <<'PY'

```
import struct,zlib,re,itertools,os
files=['secret{_}.png','operating_system.png','mario_64.png']
pat=re.compile(rb'secret\{[0-9A-Fa-f]+\}')
def paeth(a,b,c):
    p=a+b-c; pa=abs(p-a); pb=abs(p-b); pc=abs(p-c)
    return a if pa<=pb and pa<=pc else b if pb<=pc else c
def decode_png(path):
    data=open(path,'rb').read()
    assert data[:8]==b'\x89PNG\r\n\x1a\n'
    pos=8; idat=b''; chunks=[]; w=h=bd=ct=None
    while pos<len(data):
        ln=struct.unpack('>I',data[pos:pos+4])[0]; typ=data[pos+4:pos+8];
chunk=data[pos+8:pos+8+ln]
        chunks.append((typ,chunk))
        if typ==b'IHDR':
            w,h,bd,ct,comp,fil,inter=struct.unpack('>IIBBBBB',chunk)
        if typ==b'IDAT': idat+=chunk
        pos+=12+ln
        if typ==b'IEND': break
    print(path,'chunks',[(t.decode("latin1"),len(c)) for t,c in
chunks[:10]],'...', 'IHDR',w,h,bd,ct)
```

```
    raw=zlib.decompress(idat)
    channels={0:1,2:3,3:1,4:2,6:4}[ct]
    assert bd==8 and inter==0
    bpp=channels
    stride=w*channels
    rows=[]; i=0; prev=bytearray(stride)
    for y in range(h):
        ft=raw[i]; i+=1; scan=bytearray(raw[i:i+stride]); i+=stride
        out=bytearray(stride)
        for x in range(stride):
            left=out[x-bpp] if x>=bpp else 0
            up=prev[x]
            ul=prev[x-bpp] if x>=bpp else 0
            v=scan[x]
            if ft==0: out[x]=v
            elif ft==1: out[x]=(v+left)&255
            elif ft==2: out[x]=(v+up)&255
            elif ft==3: out[x]=(v+((left+up)//2))&255
            elif ft==4: out[x]=(v+paeth(left,up,ul))&255
            else: raise ValueError(ft)
        rows.append(out); prev=out
    return w,h,channels,rows,chunks
def bits_to_bytes(bits, msb=True):
    out=bytearray()
    for i in range(0,len(bits)-7,8):
        b=0
        if msb:
            for bit in bits[i:i+8]: b=(b<<1)|bit
        else:
            for j,bit in enumerate(bits[i:i+8]): b|=bit<<j
        out.append(b)
    return bytes(out)
def scan_bytes(label, data):
    for m in pat.finditer(data):
        print('FOUND',label,m.group().decode(), 'offset',m.start())
        return True
    # also hex-only inside braces with underscores? no
    return False
for f in files:
    w,h,ch,rows,chunks=decode_png(f)
    # chunk scan decompressed text
    for typ,c in chunks:
        scan_bytes(f'chunk {f} {typ}', c)
        if typ in (b'zTXt',):
            try:
                nul=c.index(b'\0'); comp=c[nul+2:]
                scan_bytes(f'zTXt {f}', zlib.decompress(comp))
            except Exception as e: pass
        if typ==b'iTXt':
            scan_bytes(f'iTXt {f}', c)
    # pixel byte streams per common channel orders
    allpix=[]
    for row in rows: allpix.extend(row)
    scan_bytes(f'raw pixels {f}', bytes(allpix))
    names=list('RGBA')[:ch] if ch==4 else list('RGB')[:ch] if ch==3 else
list(range(ch))
    channel_indices=list(range(ch))
```

```
    channel_sets=[]
    for r in range(1,ch+1):
        for comb in itertools.combinations(channel_indices,r):
channel_sets.append(comb)
    # include permutations for RGB/RGBA small
    for plane in range(4):
        for comb in channel_sets:
            orders=[comb]
            if len(comb)>1 and len(comb)<=4:
                orders=list(itertools.permutations(comb))
            for order in orders:
                bits=[]
                for row in rows:
                    for x in range(w):
                        base=x*ch
                        for cidx in order:
                            bits.append((row[base+cidx]>>plane)&1)
                label=f'{f} plane{plane} order' + ''.join(str(o) for o in order)
                b1=bits_to_bytes(bits, True); b2=bits_to_bytes(bits, False)
                scan_bytes(label+' msb', b1)
                scan_bytes(label+' lsb', b2)
                # reversed bit stream
                if plane==0:
                    rb=bits_to_bytes(bits[::-1], True)
                    scan_bytes(label+' revmsb', rb)
PY
```

- The broad LSB scan is running but slower than useful because it is trying every channel permutation on large images. I'll let it return briefly, then I'll narrow the search to more targeted byte streams if it has not produced hits.
- **Ran** pkill -f "python3 - <<"
  └ (no output)
- **Waited for background terminal** · python3 - <<'PY'

```
import struct,zlib,re,itertools,os
files=['secret{_}.png','operating_system.png','mario_64.png']
pat=re.compile(rb'secret\{[0-9A-Fa-f]+\}')
def paeth(a,b,c):
    p=a+b-c; pa=abs(p-a); pb=abs(p-b); pc=abs(p-c)
    return a if pa<=pb and pa<=pc else b if pb<=pc else c
def decode_png(path):
    data=open(path,'rb').read()
    assert data[:8]==b'\x89PNG\r\n\x1a\n'
    pos=8; idat=b''; chunks=[]; w=h=bd=ct=None
    while pos<len(data):
        ln=struct.unpack('>I',data[pos:pos+4])[0]; typ=data[pos+4:pos+8];
chunk=data[pos+8:pos+8+ln]
        chunks.append((typ,chunk))
        if typ==b'IHDR':
            w,h,bd,ct,comp,fil,inter=struct.unpack('>IIBBBBB',chunk)
        if typ==b'IDAT': idat+=chunk
        pos+=12+ln
        if typ==b'IEND': break
    print(path,'chunks',[(t.decode("latin1"),len(c)) for t,c in
chunks[:10]],'...', 'IHDR',w,h,bd,ct)
    raw=zlib.decompress(idat)
```

```
    channels={0:1,2:3,3:1,4:2,6:4}[ct]
    assert bd==8 and inter==0
    bpp=channels
    stride=w*channels
    rows=[]; i=0; prev=bytearray(stride)
    for y in range(h):
        ft=raw[i]; i+=1; scan=bytearray(raw[i:i+stride]); i+=stride
        out=bytearray(stride)
        for x in range(stride):
            left=out[x-bpp] if x>=bpp else 0
            up=prev[x]
            ul=prev[x-bpp] if x>=bpp else 0
            v=scan[x]
            if ft==0: out[x]=v
            elif ft==1: out[x]=(v+left)&255
            elif ft==2: out[x]=(v+up)&255
            elif ft==3: out[x]=(v+((left+up)//2))&255
            elif ft==4: out[x]=(v+paeth(left,up,ul))&255
            else: raise ValueError(ft)
        rows.append(out); prev=out
    return w,h,channels,rows,chunks
def bits_to_bytes(bits, msb=True):
    out=bytearray()
    for i in range(0,len(bits)-7,8):
        b=0
        if msb:
            for bit in bits[i:i+8]: b=(b<<1)|bit
        else:
            for j,bit in enumerate(bits[i:i+8]): b|=bit<<j
        out.append(b)
    return bytes(out)
def scan_bytes(label, data):
    for m in pat.finditer(data):
        print('FOUND',label,m.group().decode(), 'offset',m.start())
        return True
    # also hex-only inside braces with underscores? no
    return False
for f in files:
    w,h,ch,rows,chunks=decode_png(f)
    # chunk scan decompressed text
    for typ,c in chunks:
        scan_bytes(f'chunk {f} {typ}', c)
        if typ in (b'zTXt',):
            try:
                nul=c.index(b'\0'); comp=c[nul+2:]
                scan_bytes(f'zTXt {f}', zlib.decompress(comp))
            except Exception as e: pass
        if typ==b'iTXt':
            scan_bytes(f'iTXt {f}', c)
    # pixel byte streams per common channel orders
    allpix=[]
    for row in rows: allpix.extend(row)
    scan_bytes(f'raw pixels {f}', bytes(allpix))
    names=list('RGBA')[:ch] if ch==4 else list('RGB')[:ch] if ch==3 else
list(range(ch))
    channel_indices=list(range(ch))
    channel_sets=[]
```

```
    for r in range(1,ch+1):
        for comb in itertools.combinations(channel_indices,r):
channel_sets.append(comb)
    # include permutations for RGB/RGBA small
    for plane in range(4):
        for comb in channel_sets:
            orders=[comb]
            if len(comb)>1 and len(comb)<=4:
                orders=list(itertools.permutations(comb))
            for order in orders:
                bits=[]
                for row in rows:
                    for x in range(w):
                        base=x*ch
                        for cidx in order:
                            bits.append((row[base+cidx]>>plane)&1)
                label=f'{f} plane{plane} order' + ''.join(str(o) for o in order)
                b1=bits_to_bytes(bits, True); b2=bits_to_bytes(bits, False)
                scan_bytes(label+' msb', b1)
                scan_bytes(label+' lsb', b2)
                # reversed bit stream
                if plane==0:
                    rb=bits_to_bytes(bits[::-1], True)
                    scan_bytes(label+' revmsb', rb)
PY
```

• **Ran** python3 - <<'PY'
  │ import struct,zlib,re,itertools,os
  │ files=['secret{_}.png','operating_system.png','mario_64.png']
  │ … +108 lines
  └ (no output)

• I stopped the exhaustive run. Next I'm narrowing it to the high-probability streams first: single channels and RGB/RGBA order in bit plane 0, then only expanding if those miss.

• **Ran** pkill -f "python3 - <<"
  └ (no output)

• **Waited for background terminal** · python3 - <<'PY'

```
import struct,zlib,re,itertools,sys,os
pat=re.compile(rb'secret\{[0-9A-Fa-f]+\}')
def paeth(a,b,c):
    p=a+b-c; pa=abs(p-a); pb=abs(p-b); pc=abs(p-c)
    return a if pa<=pb and pa<=pc else b if pb<=pc else c
def decode_png(path):
    data=open(path,'rb').read(); pos=8; idat=b''; chunks=[]
    w=h=bd=ct=inter=None
    while pos < len(data):
        ln=int.from_bytes(data[pos:pos+4],'big'); typ=data[pos+4:pos+8];
chunk=data[pos+8:pos+8+ln]
        chunks.append((typ,chunk))
        if typ==b'IHDR':
w,h,bd,ct,comp,fil,inter=struct.unpack('>IIBBBBB',chunk)
        elif typ==b'IDAT': idat += chunk
        pos += 12+ln
        if typ==b'IEND': break
    ch={0:1,2:3,3:1,4:2,6:4}[ct]
    raw=zlib.decompress(idat); stride=w*ch; rows=[]; i=0; prev=bytearray(stride)
```

```
    for y in range(h):
        ft=raw[i]; i+=1; scan=raw[i:i+stride]; i+=stride; out=bytearray(stride)
        for x,v in enumerate(scan):
            left=out[x-ch] if x>=ch else 0; up=prev[x]; ul=prev[x-ch] if x>=ch
else 0
            if ft==0: out[x]=v
            elif ft==1: out[x]=(v+left)&255
            elif ft==2: out[x]=(v+up)&255
            elif ft==3: out[x]=(v+((left+up)//2))&255
            elif ft==4: out[x]=(v+paeth(left,up,ul))&255
        rows.append(out); prev=out
    return w,h,ch,rows,chunks
def bits_to_bytes_iter(rows,w,ch,channels,plane,msb=True,rev=False):
    bits=[]; out=bytearray(); append=bits.append
    row_iter=reversed(rows) if rev else rows
    for row in row_iter:
        xs=range(w-1,-1,-1) if rev else range(w)
        for x in xs:
            base=x*ch
            for c in channels:
                append((row[base+c]>>plane)&1)
                if len(bits)==8:
                    b=0
                    if msb:
                        for bit in bits: b=(b<<1)|bit
                    else:
                        for j,bit in enumerate(bits): b|=bit<<j
                    out.append(b); bits.clear()
    return bytes(out)
def scan(label,data):
    found=False
    for m in pat.finditer(data):
        print('FOUND',label,m.group().decode(),'offset',m.start()); found=True
    return found
for f in ['secret{_}.png','operating_system.png','mario_64.png']:
    w,h,ch,rows,chunks=decode_png(f)
    print('decoded',f,w,h,ch,flush=True)
    for typ,c in chunks:
        scan(f'{f} chunk {typ.decode("latin1")}', c)
        if typ==b'zTXt':
            try:
                nul=c.index(b'\0'); scan(f'{f} zTXt',
zlib.decompress(c[nul+2:]))
            except Exception: pass
    orders=[]
    for c in range(ch): orders.append((c,))
    if ch>=3: orders += [(0,1,2),(2,1,0)]
    if ch==4: orders += [(0,1,2,3),(3,2,1,0),(3,),(0,1,2)]
    seen=set(); orders=[o for o in orders if not (o in seen or seen.add(o))]
    for plane in range(8):
        for order in orders:
            for msb in (True,False):
                data=bits_to_bytes_iter(rows,w,ch,order,plane,msb,False)
                scan(f'{f} plane{plane} order{order} {"msb" if msb else "lsb"}',
data)
            if plane==0:
                data=bits_to_bytes_iter(rows,w,ch,order,plane,True,True)
```

```
            scan(f'{f} plane{plane} order{order} rev msb', data)
PY
```

• **Ran** python3 - <<'PY'

```
  │ import struct,zlib,re,itertools,sys,os
  │ pat=re.compile(rb'secret\{[0-9A-Fa-f]+\}')
  │ … +84 lines
  └ decoded secret{_}.png 1272 686 4
```

• **Ran** python3 -c "import numpy; print(numpy.__version__)"

```
  └ Traceback (most recent call last):
      File "<string>", line 1, in <module>
    ModuleNotFoundError: No module named 'numpy'
```

• **Ran** pkill -f "python3 - <<"

```
  └ (no output)
```

• **Waited for background terminal** · python3 - <<'PY'

```
import struct,zlib,re,itertools
pat=re.compile(rb'secret\{[0-9A-Fa-f]+\}')
def paeth(a,b,c):
    p=a+b-c; pa=abs(p-a); pb=abs(p-b); pc=abs(p-c)
    return a if pa<=pb and pa<=pc else b if pb<=pc else c
def decode(path):
    d=open(path,'rb').read(); pos=8; idat=b''; w=h=ct=bd=None; chunks=[]
    while pos<len(d):
        n=int.from_bytes(d[pos:pos+4],'big'); t=d[pos+4:pos+8];
c=d[pos+8:pos+8+n]
        chunks.append((t,c))
        if t==b'IHDR': w,h,bd,ct,_,_,inter=struct.unpack('>IIBBBBB',c)
        elif t==b'IDAT': idat+=c
        pos+=n+12
    ch={2:3,6:4}[ct]
    raw=zlib.decompress(idat); stride=w*ch; prev=bytearray(stride);
out=bytearray(w*h*ch); ri=0; oi=0
    for y in range(h):
        ft=raw[ri]; ri+=1; scan=raw[ri:ri+stride]; ri+=stride;
row=bytearray(stride)
        for i,v in enumerate(scan):
            left=row[i-ch] if i>=ch else 0; up=prev[i]; ul=prev[i-ch] if i>=ch
else 0
            if ft==0: row[i]=v
            elif ft==1: row[i]=(v+left)&255
            elif ft==2: row[i]=(v+up)&255
            elif ft==3: row[i]=(v+((left+up)//2))&255
            else: row[i]=(v+paeth(left,up,ul))&255
        out[oi:oi+stride]=row; oi+=stride; prev=row
    return w,h,ch,bytes(out),chunks
def pack_from_values(vals,plane=0,msb=True):
    out=bytearray(); b=0; n=0
    if msb:
        for v in vals:
            b=(b<<1)|((v>>plane)&1); n+=1
            if n==8: out.append(b); b=0; n=0
    else:
        for v in vals:
            b|=((v>>plane)&1)<<n; n+=1
            if n==8: out.append(b); b=0; n=0
    return bytes(out)
def scan(label,data):
    for m in pat.finditer(data):
```

```
print('FOUND',label,m.group().decode(),m.start())
for f in ['secret{_}.png','operating_system.png','mario_64.png']:
    w,h,ch,pix,chunks=decode(f); print('decoded',f,w,h,ch)
    for t,c in chunks:
        scan(f'{f} chunk {t}',c)
        if t==b'zTXt':
            try:
                nul=c.index(b'\0'); scan(f'{f} zTXt',
zlib.decompress(c[nul+2:]))
            except Exception: pass
    orders=[]
    for c in range(ch): orders.append((c,))
    if ch>=3: orders += [(0,1,2),(2,1,0)]
    if ch==4: orders += [(0,1,2,3),(3,2,1,0)]
    for plane in range(2):
      for order in orders:
        vals=[]
        if len(order)==1:
            vals=pix[order[0]::ch]
        else:
            vals=bytearray()
            # common row-major channel interleave
            for i in range(0,len(pix),ch):
                for c in order: vals.append(pix[i+c])
        for msb in (True,False): scan(f'{f} p{plane} o{order} {msb}',
pack_from_values(vals,plane,msb))
PY
```

• **Ran** python3 - <<'PY'
  │ import struct,zlib,re,itertools
  │ pat=re.compile(rb'secret\{[0-9A-Fa-f]+\}')
  │ … +73 lines
  └ (no output)

— Worked for 1m 26s —

• The standard-library PNG decoder is still too slow in this environment for broad scans. Since macOS sips is available, I'm converting the images to simple BMP files under /tmp and scanning those raw pixels instead, including the JPEG.

• **Ran** sips -s format bmp operating_system.png --out /tmp/operating_system.bmp
  └ /Users/abdullahalwakeel/Documents/GitHub/tb3_tasks/steganography/operating_system.png
    /private/tmp/operating_system.bmp

• **Ran** sips -s format bmp mario_64.png --out /tmp/mario_64.bmp
  └ /Users/abdullahalwakeel/Documents/GitHub/tb3_tasks/steganography/mario_64.png
    /private/tmp/mario_64.bmp

• **Ran** sips -s format bmp airplane.jpg --out /tmp/airplane.bmp
  └ sysctlbyname for kern.hv_vmm_present failed with status -1/Users/abdullahalwakeel/Documents/GitHub/tb3_tasks/steganography/airplane.jpg
    /private/tmp/airplane.bmp

• **Ran** sips -s format bmp secret\{_\}.png --out /tmp/secret.bmp
  └ /Users/abdullahalwakeel/Documents/GitHub/tb3_tasks/steganography/secret{_}.png
    /private/tmp/secret.bmp

• **Ran** file /tmp/airplane.bmp /tmp/secret.bmp /tmp/operating_system.bmp /tmp/
  │ mario_64.bmp
  └ /tmp/airplane.bmp: PC bitmap, Windows 3.x format, 1400 x -1434 x 24, image size 6022800, resolution 11811 x 11811 px/m, cbSize 6022854, bits

… +5 lines
/tmp/mario_64.bmp: PC bitmap, Windows 98/2000 and newer format, 640 x -480 x 32, cbSize 1228938, bits offset 138

• **Ran** pkill -f "python3 - <<"
└ (no output)

• **Waited for background terminal** · python3 - <<'PY'

```
import struct,re,itertools,os
pat=re.compile(rb'secret\{[0-9A-Fa-f]+\}')
files={'airplane':'/tmp/airplane.bmp','secret':'/tmp/secret.bmp','os':'/tmp/
operating_system.bmp','mario':'/tmp/mario_64.bmp'}
def read_bmp(path):
    d=open(path,'rb').read(); off=int.from_bytes(d[10:14],'little');
dib=int.from_bytes(d[14:18],'little')
    w=int.from_bytes(d[18:22],'little',signed=True);
h=int.from_bytes(d[22:26],'little',signed=True);
bpp=int.from_bytes(d[28:30],'little')
    topdown=h<0; h=abs(h); n=bpp//8; stride=((w*bpp+31)//32)*4
    rows=[]
    for y in range(h):
        src_y=y if topdown else h-1-y
        row=d[off+src_y*stride:off+src_y*stride+w*n]
        rows.append(row)
    pix=b''.join(rows)
    return w,h,n,pix,d
def pack(vals,plane,msb=True):
    out=bytearray(); b=0; n=0
    if msb:
        for v in vals:
            b=(b<<1)|((v>>plane)&1); n+=1
            if n==8: out.append(b); b=0; n=0
    else:
        for v in vals:
            b|=((v>>plane)&1)<<n; n+=1
            if n==8: out.append(b); b=0; n=0
    return bytes(out)
def scan(label,data):
    for m in pat.finditer(data):
print('FOUND',label,m.group().decode(),m.start())
for name,path in files.items():
    w,h,n,pix,d=read_bmp(path); print('bmp',name,w,h,n,len(pix),flush=True)
    scan(name+' fullbmp',d); scan(name+' pixels',pix)
    # BMP pixel byte order is B,G,R,A. Try BGR, RGB, each channel, BGRA/RGBA.
    orders=[(0,),(1,),(2,)]
    if n==4: orders.append((3,))
    orders += [(0,1,2),(2,1,0)]
    if n==4: orders += [(0,1,2,3),(2,1,0,3),(3,2,1,0)]
    for plane in range(4):
        for order in orders:
            if len(order)==1: vals=pix[order[0]::n]
            else:
                vals=bytearray()
                for i in range(0,len(pix),n):
                    for c in order: vals.append(pix[i+c])
            for msb in (True,False): scan(f'{name} p{plane} o{order} {msb}',
pack(vals,plane,msb))
PY
```

• **Ran** python3 - <<'PY'

```
import struct,re,itertools,os
pat=re.compile(rb'secret\{[0-9A-Fa-f]+\}')
… +53 lines
(no output)
```

- **Ran** which stegseek
  └ (no output)
- **Ran** which outguess
  └ (no output)
- **Ran** which steghide
  └ (no output)
- **Ran** which foremost
  └ (no output)

- The raw BMP scan is also too slow in Python for the large JPEG-derived image. I'm going to compile a small local C scanner so the bit-plane permutations run at native speed.
- **Added** scan_lsb.c (+130 -0)

```
+#include <stdio.h>
+#include <stdlib.h>
+#include <stdint.h>
+#include <string.h>
+#include <ctype.h>
+
+static int is_hex_secret(const unsigned char *buf, size_t n, size_t pos, size_t *end) {
+    const char *p = "secret{";
+    if (pos + 8 >= n) return 0;
+    if (memcmp(buf + pos, p, 7) != 0) return 0;
+    size_t i = pos + 7;
+    int digits = 0;
+    while (i < n && isxdigit(buf[i])) {
+        i++;
+        digits++;
+    }
+    if (digits > 0 && i < n && buf[i] == '}') {
+        *end = i + 1;
+        return 1;
+    }
+    return 0;
+}
+
+static void scan_bytes(const char *label, const unsigned char *buf, size_t n) {
+    for (size_t i = 0; i < n; i++) {
+        size_t end = 0;
+        if (is_hex_secret(buf, n, i, &end)) {
+            printf("FOUND %s offset %zu: ", label, i);
+            fwrite(buf + i, 1, end - i, stdout);
+            putchar('\n');
+        }
+    }
+}
+
+static uint32_t le32(const unsigned char *p) {
+    return (uint32_t)p[0] | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16) | ((uint32_t)p[3] << 24);
```

```
+}
+
+static int32_t sle32(const unsigned char *p) {
+    return (int32_t)le32(p);
+}
+
+static uint16_t le16(const unsigned char *p) {
+    return (uint16_t)p[0] | ((uint16_t)p[1] << 8);
+}
+
+static void pack_and_scan(const char *name, const unsigned char *pix, int w, int h, int nchan,
+                          const int *order, int olen, int plane, int msb) {
+    size_t max_bits = (size_t)w * (size_t)h * (size_t)olen;
+    size_t outn = max_bits / 8;
+    unsigned char *out = calloc(outn ? outn : 1, 1);
+    if (!out) return;
+    size_t oi = 0;
+    int bitn = 0;
+    unsigned char b = 0;
+    for (int y = 0; y < h; y++) {
+        const unsigned char *row = pix + (size_t)y * (size_t)w * (size_t)nchan;
+        for (int x = 0; x < w; x++) {
+            const unsigned char *px = row + (size_t)x * (size_t)nchan;
+            for (int j = 0; j < olen; j++) {
+                int bit = (px[order[j]] >> plane) & 1;
+                if (msb) b = (unsigned char)((b << 1) | bit);
+                else b = (unsigned char)(b | (bit << bitn));
+                bitn++;
+                if (bitn == 8) {
+                    out[oi++] = b;
+                    b = 0;
+                    bitn = 0;
+                }
+            }
+        }
+    }
+    char label[256];
+    snprintf(label, sizeof(label), "%s p%d o", name, plane);
+    size_t len = strlen(label);
+    for (int j = 0; j < olen && len + 4 < sizeof(label); j++) {
+        len += snprintf(label + len, sizeof(label) - len, "%d", order[j]);
+    }
+    snprintf(label + strlen(label), sizeof(label) - strlen(label), " %s", msb ? "msb" : "lsb");
+    scan_bytes(label, out, oi);
+    free(out);
+}
+
+int main(int argc, char **argv) {
+    for (int ai = 1; ai < argc; ai++) {
+        const char *path = argv[ai];
+        FILE *f = fopen(path, "rb");
+        if (!f) { perror(path); continue; }
```

```
 89 +        fseek(f, 0, SEEK_END);
 90 +        long flen = ftell(f);
 91 +        fseek(f, 0, SEEK_SET);
 92 +        unsigned char *d = malloc((size_t)flen);
 93 +        if (!d || fread(d, 1, (size_t)flen, f) != (size_t)flen) { fclos
    e(f); free(d); continue; }
 94 +        fclose(f);
 95 +        scan_bytes(path, d, (size_t)flen);
 96 +        if (flen < 54 || d[0] != 'B' || d[1] != 'M') { free(d); continu
    e; }
 97 +        uint32_t off = le32(d + 10);
 98 +        int32_t w = sle32(d + 18);
 99 +        int32_t sh = sle32(d + 22);
100 +        int h = sh < 0 ? -sh : sh;
101 +        int topdown = sh < 0;
102 +        int bpp = le16(d + 28);
103 +        int nchan = bpp / 8;
104 +        if (w <= 0 || h <= 0 || (nchan != 3 && nchan != 4)) { free(d);
    continue; }
105 +        size_t stride = (((size_t)w * (size_t)bpp + 31) / 32) * 4;
106 +        unsigned char *pix = malloc((size_t)w * (size_t)h * (size_t)nch
    an);
107 +        if (!pix) { free(d); continue; }
108 +        for (int y = 0; y < h; y++) {
109 +            int sy = topdown ? y : h - 1 - y;
110 +            memcpy(pix + (size_t)y * (size_t)w * (size_t)nchan, d + off
     + (size_t)sy * stride, (size_t)w * (size_t)nchan);
111 +        }
112 +        printf("SCAN %s %dx%d %dch\n", path, w, h, nchan);
113 +        int orders[9][4] = {
114 +            {0,-1,-1,-1}, {1,-1,-1,-1}, {2,-1,-1,-1}, {3,-1,-1,-1},
115 +            {0,1,2,-1}, {2,1,0,-1}, {0,1,2,3}, {2,1,0,3}, {3,2,1,0}
116 +        };
117 +        int lens[9] = {1,1,1,1,3,3,4,4,4};
118 +        int count = nchan == 3 ? 6 : 9;
119 +        for (int plane = 0; plane < 8; plane++) {
120 +            for (int oi = 0; oi < count; oi++) {
121 +                if (orders[oi][0] >= nchan) continue;
122 +                pack_and_scan(path, pix, w, h, nchan, orders[oi], lens[
    oi], plane, 1);
123 +                pack_and_scan(path, pix, w, h, nchan, orders[oi], lens[
    oi], plane, 0);
124 +            }
125 +        }
126 +        free(pix);
127 +        free(d);
128 +    }
129 +    return 0;
130 +}
```

• **Ran** cc -O2 scan_lsb.c -o /tmp/scan_lsb
└ (no output)

• **Ran** /tmp/scan_lsb /tmp/airplane.bmp /tmp/secret.bmp /tmp/
│ operating_system.bmp /tmp/mario_64.bmp
└ SCAN /tmp/airplane.bmp 1400x1434 3ch
SCAN /tmp/secret.bmp 1272x686 4ch
SCAN /tmp/operating_system.bmp 1152x864 4ch
SCAN /tmp/mario_64.bmp 640x480 4ch

FOUND /tmp/mario_64.bmp p0 o210 msb offset 0: secret{aaff4578}

• **Waited for background terminal**

• The native scanner found one byte-stream LSB secret in mario_64.png: secret{aaff4578}. The other three are likely using a different hiding style, so I'm switching to visual bit-plane extraction next: if the message is drawn into low bits, it will appear as text in a generated plane image.

• **Waited for background terminal** · python3 - <<'PY'

```
import os,struct
files={'airplane':'/tmp/airplane.bmp','secret':'/tmp/secret.bmp','os':'/tmp/
operating_system.bmp','mario':'/tmp/mario_64.bmp'}
os.makedirs('/tmp/planes',exist_ok=True)
def read_bmp(path):
 d=open(path,'rb').read(); off=int.from_bytes(d[10:14],'little');
w=int.from_bytes(d[18:22],'little',signed=True);
sh=int.from_bytes(d[22:26],'little',signed=True);
bpp=int.from_bytes(d[28:30],'little'); h=abs(sh); top=sh<0; n=bpp//8;
stride=((w*bpp+31)//32)*4; rows=[]
 for y in range(h):
  sy=y if top else h-1-y
  rows.append(d[off+sy*stride:off+sy*stride+w*n])
 return w,h,n,rows
for name,path in files.items():
 w,h,n,rows=read_bmp(path)
 for plane in range(2):
  for c in range(n):
   out=bytearray(f'P5\n{w} {h}\n255\n'.encode())
   for row in rows:
    for x in range(w):
     out.append(255 if ((row[x*n+c]>>plane)&1) else 0)
   fn=f'/tmp/planes/{name}_p{plane}_c{c}.pgm'
   open(fn,'wb').write(out)
 print(name,w,h,n)
PY
```

• **Ran** python3 - <<'PY'

```
│ import os,struct
│ files={'airplane':'/tmp/airplane.bmp','secret':'/tmp/secret.bmp','os':'/tmp/
│ … +24 lines
└ airplane 1400 1434 3
  secret 1272 686 4
  os 1152 864 4
  mario 640 480 4
```

• **Ran** sips -s format png /tmp/planes/secret_p0_c3.pgm --out /tmp/planes/

```
│ secret_p0_c3.png
└ /private/tmp/planes/secret_p0_c3.pgm
    /private/tmp/planes/secret_p0_c3.png
```

• **Viewed Image**

```
└ /tmp/planes/secret_p0_c3.png
```

• **Waited for background terminal**

• **Waited for background terminal** · python3 - <<'PY'

```
import glob,os,re
from pathlib import Path
os.makedirs('/tmp/contact',exist_ok=True)
for name in ['airplane','secret','os','mario']:
    files=sorted(glob.glob(f'/tmp/planes/{name}_p*_c*.pgm'))
    imgs=[]
```

```
    for fn in files:
        data=open(fn,'rb').read()
        parts=data.split(b'\n',3); magic=parts[0];
w,h=map(int,parts[1].split()); maxv=int(parts[2]); pix=parts[3]
        label=os.path.basename(fn).replace('.pgm','')
        imgs.append((label,w,h,pix))
    thumbw=240; cols=4; rows=(len(imgs)+cols-1)//cols; labelh=20
    thumbh=max(1,int(imgs[0][2]*thumbw/imgs[0][1]))
    W=cols*thumbw; H=rows*(thumbh+labelh)
    out=bytearray([230,230,230])*(W*H)
    # no text drawing, encode labels in filename ordering; use black separator
bars
    for idx,(label,w,h,pix) in enumerate(imgs):
        ox=(idx%cols)*thumbw; oy=(idx//cols)*(thumbh+labelh)+labelh
        for yy in range(thumbh):
            sy=yy*h//thumbh
            for xx in range(thumbw):
                sx=xx*w//thumbw; v=pix[sy*w+sx]
                p=((oy+yy)*W+ox+xx)*3
                out[p:p+3]=bytes([v,v,v])
        # simple label bars: plane/channel encoded as black ticks from filename
        m=re.search(r'_p(\d+)_c(\d+)',label); pnum=int(m.group(1));
cnum=int(m.group(2))
        for yy in range(2,6):
            for xx in range(4,4+20*(pnum+1)):
                q=((oy-labelh+yy)*W+ox+xx)*3; out[q:q+3]=b'\0\0\0'
        for yy in range(10,14):
            for xx in range(4,4+20*(cnum+1)):
                q=((oy-labelh+yy)*W+ox+xx)*3; out[q:q+3]=b'\0\0\0'
    ppm=f'/tmp/contact/{name}.ppm'; open(ppm,'wb').write(f'P6\n{W}
{H}\n255\n'.encode()+out)
PY
sips -s format png /tmp/contact/airplane.ppm --out /tmp/contact/airplane.png
sips -s format png /tmp/contact/secret.ppm --out /tmp/contact/secret.png
sips -s format png /tmp/contact/os.ppm --out /tmp/contact/os.png
sips -s format png /tmp/contact/mario.ppm --out /tmp/contact/mario.png
```

- **Ran** python3 - <<'PY'
  - import glob,os,re
  - from pathlib import Path
  - … +41 lines
  - /private/tmp/contact/airplane.ppm
    /private/tmp/contact/airplane.png
  - … +4 lines
  - /private/tmp/contact/mario.ppm
    /private/tmp/contact/mario.png
- **Viewed Image**
  - /tmp/contact/secret.png
- **Ran** sips -s format png /tmp/planes/secret_p0_c0.pgm --out /tmp/planes/
  - secret_p0_c0.png
  - /private/tmp/planes/secret_p0_c0.pgm
    /private/tmp/planes/secret_p0_c0.png
- **Viewed Image**
  - /tmp/planes/secret_p0_c0.png
- **Viewed Image**
  - /tmp/contact/os.png
- **Ran** sips -s format png /tmp/planes/os_p0_c1.pgm --out /tmp/planes/os_p0_c1.png
  - /private/tmp/planes/os_p0_c1.pgm

/private/tmp/planes/os_p0_c1.png

• **Ran** python3 - <<'PY'

```
│ fn='/tmp/planes/os_p0_c1.pgm'; data=open(fn,'rb').read();
│ p=data.split(b'\n',3); w,h=map(int,p[1].split()); pix=p[3]
│ … +12 lines
└ /private/tmp/planes/os_secret_zoom.pgm
  /private/tmp/planes/os_secret_zoom.png
```

• **Viewed Image**

└ /tmp/planes/os_secret_zoom.png

• **Viewed Image**

└ /tmp/contact/airplane.png

• **Ran** python3 - <<'PY'

```
│ from pathlib import Path
│ for f in
│ … +10 lines
└ airplane.jpg size 259208
  EOI 259206 tail 0 b''
  … +4 lines
  mario_64.png size 810937
  IEND 810929 tail 0 b''
```

• **Viewed Image**

└ airplane.jpg

— Worked for 1m 37s ——

• The JPEG does not have appended data or a raw secret{} string, and its decoded LSB planes look like JPEG noise. I'm checking visual-channel transforms now, since JPEG stego in these tasks is often hidden by contrast/channel differences rather than raw appended bytes.

• **Waited for background terminal** · python3 - <<'PY'

```
# Generate channel, difference, and threshold-enhanced views for airplane BMP.
d=open('/tmp/airplane.bmp','rb').read(); off=int.from_bytes(d[10:14],'little');
w=int.from_bytes(d[18:22],'little',signed=True);
sh=int.from_bytes(d[22:26],'little',signed=True); h=abs(sh); top=sh<0;
bpp=int.from_bytes(d[28:30],'little'); n=bpp//8; stride=((w*bpp+31)//32)*4
rows=[]
for y in range(h):
 sy=y if top else h-1-y; rows.append(d[off+sy*stride:off+sy*stride+w*n])
def write(name, func):
 out=bytearray(f'P5\n{w} {h}\n255\n'.encode())
 vals=[]
 for row in rows:
  for x in range(w):
   b,g,r=row[x*n],row[x*n+1],row[x*n+2]
   vals.append(func(r,g,b))
 mn=min(vals); mx=max(vals)
 for v in vals:
  if mx>mn: v=(v-mn)*255//(mx-mn)
  else: v=0
  out.append(max(0,min(255,v)))
 open(f'/tmp/planes/air_{name}.pgm','wb').write(out)
write('r',lambda r,g,b:r); write('g',lambda r,g,b:g); write('b',lambda r,g,b:b)
write('rg',lambda r,g,b:r-g+255); write('rb',lambda r,g,b:r-b+255);
write('gb',lambda r,g,b:g-b+255)
write('maxmin',lambda r,g,b:max(r,g,b)-min(r,g,b)); write('lum',lambda r,g,b:
(r*30+g*59+b*11)//100)
PY
sips -s format png /tmp/planes/air_r.pgm --out /tmp/planes/air_r.png
sips -s format png /tmp/planes/air_g.pgm --out /tmp/planes/air_g.png
```

```
sips -s format png /tmp/planes/air_b.pgm --out /tmp/planes/air_b.png
sips -s format png /tmp/planes/air_rg.pgm --out /tmp/planes/air_rg.png
sips -s format png /tmp/planes/air_rb.pgm --out /tmp/planes/air_rb.png
sips -s format png /tmp/planes/air_gb.pgm --out /tmp/planes/air_gb.png
sips -s format png /tmp/planes/air_maxmin.pgm --out /tmp/planes/air_maxmin.png
sips -s format png /tmp/planes/air_lum.pgm --out /tmp/planes/air_lum.png
```

• **Ran** python3 - <<'PY'
  │ # Generate channel, difference, and threshold-enhanced views for airplane
  │ BMP.
  │ … +37 lines
  └ /private/tmp/planes/air_r.pgm
    /private/tmp/planes/air_r.png
  … +12 lines
  /private/tmp/planes/air_lum.pgm
    /private/tmp/planes/air_lum.png

• **Viewed Image**
  └ /tmp/planes/air_gb.png

• **Viewed Image**
  └ /tmp/planes/air_rb.png

• **Ran** which djpeg
  └ /Users/abdullahalwakeel/anaconda3/bin/djpeg

• **Ran** which jpegtran
  └ /Users/abdullahalwakeel/anaconda3/bin/jpegtran

• **Ran** which jpegtopnm
  └ (no output)

• **Ran** which identify
  └ (no output)

• **Ran** djpeg -verbose -verbose airplane.jpg > /tmp/airplane.ppm
  └ Independent JPEG Group's DJPEG, version 9b 17-Jan-2016
  Copyright (C) 2016, Thomas G. Lane, Guido Vollbeding
  … +90 lines
    Ss=1, Se=63, Ah=1, Al=0
  End Of Image

• **Ran** python3 -c "import scipy; print('scipy ok')"
  └ Traceback (most recent call last):
    File "<string>", line 1, in <module>
  ModuleNotFoundError: No module named 'scipy'

• **Ran** python3 -c "import jpegio; print('jpegio ok')"
  └ Traceback (most recent call last):
    File "<string>", line 1, in <module>
  ModuleNotFoundError: No module named 'jpegio'

• **Ran** python3 -c "import PIL; print('PIL ok')"
  └ Traceback (most recent call last):
    File "<string>", line 1, in <module>
  ModuleNotFoundError: No module named 'PIL'

• **Ran** python3 -c "import skimage; print('skimage ok')"
  └ Traceback (most recent call last):
    File "<string>", line 1, in <module>
  ModuleNotFoundError: No module named 'skimage'

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python -c "import PIL; print('PIL
  │ ok')"
  └ PIL ok

• **Waited for background terminal** · /Users/abdullahalwakeel/anaconda3/bin/python
-c "import numpy; print('numpy ok')"

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python -c "import numpy;
  │ print('numpy ok')"
  └ numpy ok

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python -c "import cv2; print('cv2
│ ok')"
└ cv2 ok
• **Waited for background terminal** · /Users/abdullahalwakeel/anaconda3/bin/python
-c "import scipy; print('scipy ok')"
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python -c "import scipy;
│ print('scipy ok')"
└ scipy ok
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python -c "import jpegio;
│ print('jpegio ok')"
└ Traceback (most recent call last):
  File "<string>", line 1, in <module>
 ModuleNotFoundError: No module named 'jpegio'
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python -c "import stegano;
│ print('stegano ok')"
└ Traceback (most recent call last):
  File "<string>", line 1, in <module>
 ModuleNotFoundError: No module named 'stegano'
• **Waited for background terminal**
• **Waited for background terminal** · /Users/abdullahalwakeel/anaconda3/bin/python
- <<'PY'

```
from PIL import Image
import numpy as np,re,itertools
pat=re.compile(rb'secret\{[0-9A-Fa-f]+\}')
def packbits(bits, bitorder):
    return np.packbits(bits.astype(np.uint8), bitorder=bitorder).tobytes()
for f in ['airplane.jpg','secret{_}.png','operating_system.png','mario_64.png']:
    arr=np.array(Image.open(f).convert('RGBA'))
    h,w,ch=arr.shape
    print('img',f,arr.shape)
    orders=[]
    for r in range(1,5):
        for comb in itertools.permutations(range(4),r):
            # limit to single, rgb perms, rgba perms
            if r==1 or (r==3 and set(comb)==={0,1,2}) or (r==4):
orders.append(comb)
    for plane in range(4):
        for order in orders:
            bits=((arr[:,:,order].reshape(-1) >> plane) & 1).astype(np.uint8)
            for bo in ['big','little']:
                data=packbits(bits,bo)
                m=pat.search(data)
                if m:
print('FOUND',f,plane,order,bo,m.group().decode(),m.start())
PY
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image
│ import numpy as np,re,itertools
│ … +25 lines
└ img airplane.jpg (1434, 1400, 4)
 img secret{_}.png (686, 1272, 4)
 img operating_system.png (864, 1152, 4)
 img mario_64.png (480, 640, 4)
 FOUND mario_64.png 0 (0, 1, 2) big secret{aaff4578} 0
• **Ran** sips -g all airplane.jpg
└ /Users/abdullahalwakeel/Documents/GitHub/tb3_tasks/steganography/
 airplane.jpg

… +12 lines
creation: 2026:04:16 11:24:47
software: GIMP 2.10.22

• **Ran** strings -a airplane.jpg | rg -o "[0-9a-fA-F]{8,}" | sort -u | head -100
└ 00000000
046ba39e
… +10 lines
ed149e201997
eee6ab09

• **Ran** pkill -f "find /Users/abdullahalwakeel -name steghide"
└ sysmon request failed with error: sysmond service not found
pkill: Cannot get process list

• **Waited for background terminal** · find /Users/abdullahalwakeel -name steghide -type f 2>/dev/null | head

— Worked for 2m 09s —

• For airplane.jpg, I'm moving to JPEG coefficient extraction. The file is progressive JPEG, and the usual pixel-domain scans won't recover coefficient-domain stego, so I'm going to scan DCT coefficient LSB streams through libjpeg.

• **Added** scan_jpeg_coeff.c (+98 -0)

```
+#include <stdio.h>
+#include <stdlib.h>
+#include <string.h>
+#include <ctype.h>
+#include <jpeglib.h>
+
+static int match_secret(const unsigned char *buf, size_t n, size_t pos, size_t *end) {
+    if (pos + 8 >= n || memcmp(buf + pos, "secret{", 7) != 0) return 0;
+    size_t i = pos + 7;
+    int digits = 0;
+    while (i < n && isxdigit(buf[i])) { i++; digits++; }
+    if (digits && i < n && buf[i] == '}') { *end = i + 1; return 1; }
+    return 0;
+}
+
+static void scan(const char *label, const unsigned char *buf, size_t n) {
+    for (size_t i = 0; i < n; i++) {
+        size_t end = 0;
+        if (match_secret(buf, n, i, &end)) {
+            printf("FOUND %s offset %zu: ", label, i);
+            fwrite(buf + i, 1, end - i, stdout);
+            putchar('\n');
+        }
+    }
+}
+
+static void append_bit(unsigned char **out, size_t *n, size_t *cap, int *bitn, unsigned char *cur, int bit, int msb) {
+    if (*n >= *cap) {
+        *cap = *cap ? *cap * 2 : 4096;
+        *out = realloc(*out, *cap);
+        if (!*out) exit(2);
+    }
+    if (msb) *cur = (unsigned char)((*cur << 1) | (bit & 1));
+    else *cur = (unsigned char)(*cur | ((bit & 1) << *bitn));
```

```
35 +    (*bitn)++;
36 +    if (*bitn == 8) {
37 +        (*out)[(*n)++] = *cur;
38 +        *cur = 0;
39 +        *bitn = 0;
40 +    }
41 +}
42 +
43 +static void build_stream(struct jpeg_decompress_struct *cinfo, jvirt_bar
   ray_ptr *coef_arrays,
44 +                          int mode, int msb, unsigned char **out, size_t
   *outn) {
45 +    size_t cap = 0, n = 0;
46 +    unsigned char *buf = NULL, cur = 0;
47 +    int bitn = 0;
48 +    for (int ci = 0; ci < cinfo->num_components; ci++) {
49 +        jpeg_component_info *compptr = cinfo->comp_info + ci;
50 +        for (JDIMENSION by = 0; by < compptr->height_in_blocks; by++) {
51 +            JBLOCKARRAY row = (*cinfo->mem->access_virt_barray)
52 +                ((j_common_ptr)cinfo, coef_arrays[ci], by, 1, FALSE);
53 +            for (JDIMENSION bx = 0; bx < compptr->width_in_blocks; bx++)
    {
54 +                JCOEFPTR block = row[0][bx];
55 +                int start = (mode == 2) ? 0 : 1;
56 +                for (int k = start; k < DCTSIZE2; k++) {
57 +                    int v = block[k];
58 +                    if (mode == 0 && (v == 0 || v == 1 || v == -1)) cont
   inue; /* jsteg-like */
59 +                    if (mode == 1 && v == 0) continue;
60 +                    int bit = abs(v) & 1;
61 +                    append_bit(&buf, &n, &cap, &bitn, &cur, bit, msb);
62 +                }
63 +            }
64 +        }
65 +    }
66 +    *out = buf;
67 +    *outn = n;
68 +}
69 +
70 +int main(int argc, char **argv) {
71 +    for (int ai = 1; ai < argc; ai++) {
72 +        FILE *in = fopen(argv[ai], "rb");
73 +        if (!in) { perror(argv[ai]); continue; }
74 +        struct jpeg_decompress_struct cinfo;
75 +        struct jpeg_error_mgr jerr;
76 +        cinfo.err = jpeg_std_error(&jerr);
77 +        jpeg_create_decompress(&cinfo);
78 +        jpeg_stdio_src(&cinfo, in);
79 +        jpeg_read_header(&cinfo, TRUE);
80 +        jvirt_barray_ptr *coef_arrays = jpeg_read_coefficients(&cinfo);
81 +        printf("SCAN %s components=%d\n", argv[ai], cinfo.num_components
   );
82 +        for (int mode = 0; mode < 3; mode++) {
83 +            for (int msb = 0; msb < 2; msb++) {
84 +                unsigned char *buf = NULL;
85 +                size_t n = 0;
86 +                build_stream(&cinfo, coef_arrays, mode, msb, &buf, &n);
```

```
+                char label[128];
+                snprintf(label, sizeof(label), "%s mode%d %s", argv[ai],
    mode, msb ? "msb" : "lsb");
+                scan(label, buf, n);
+                free(buf);
+            }
+        }
+        jpeg_finish_decompress(&cinfo);
+        jpeg_destroy_decompress(&cinfo);
+        fclose(in);
+    }
+    return 0;
+}
```

• **Ran** cc -O2 scan_jpeg_coeff.c -o /tmp/scan_jpeg_coeff -I/Users/
abdullahalwakeel/anaconda3/include -L/Users/abdullahalwakeel/anaconda3/lib
-ljpeg
└ (no output)

• **Ran** /tmp/scan_jpeg_coeff airplane.jpg
└ dyld[73582]: Library not loaded: @rpath/libjpeg.9.dylib
Referenced from: <02E6EC5F-183D-31B9-83FD-F402546B02B6> /private/tmp/
scan_jpeg_coeff
Reason: no LC_RPATH's found

• **Ran** DYLD_LIBRARY_PATH=/Users/abdullahalwakeel/anaconda3/lib /tmp/
scan_jpeg_coeff airplane.jpg
└ dyld[73603]: symbol '__ZNKSt3__119bad_expected_accessIvE4whatEv' missing
from root that overrides /usr/lib/libc++.1.dylib. Use of that symbol in /
… +9 lines
0xBAD4007.
SCAN airplane.jpg components=3

• **Ran** which brew
└ /usr/local/bin/brew

• **Ran** which apt-get
└ (no output)

• **Ran** which gem
└ /usr/bin/gem

• **Ran** find /Users/abdullahalwakeel -name steghide -type f 2>/dev/null | head
└ (no output)

• **Ran** brew list steghide
└ Warning: cask_tap_migrations.jws.json: update failed, falling back to cached
version.
… +13 lines
Library/Caches/Homebrew/api/cask_tap_migrations.jws.json
Error: No available formula with the name "steghide".

• **Waited for background terminal**

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
from pathlib import Path
import re, collections
… +25 lines
└ 0 0xe0 len 16 at 2 b'JFIF\x00\x01\x01\x01\x01,\x01,\x00\x00'
1 0xe1 len 13064 at 20
… +47 lines
b'\x7f\x88\x17\xa5>\x00\xf1\xfa\xe5\xfa\x8e\xfck\x9en\xed\xc1Y\xff\x00\x133i
\xe6\xe0u0\xf8\x9fi'

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
from PIL import Image
im=Image.open('airplane.jpg')
… +5 lines

└ {34665: 148, 40961: 1, 305: 'GIMP 2.10.22', 274: 1, 306: '2026:04:16
11:24:47', 296: 2, 282: 300.0, 283: 300.0}
… +7 lines
283 300.0
thumbnail True

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from pathlib import Path
│ import struct,re
│ … +36 lines
└ 13062 b'Exif\x00\x00II*\x00'
b'II*\x00\x08\x00\x00\x00'
… +11 lines
b'\x89PNG' []
b'secret' []

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from pathlib import Path
│ exif=Path('/tmp/air_exif.bin').read_bytes()
│ … +7 lines
└ 280 13061 12781
/tmp/air_exif_thumb.jpg: JPEG image data, JFIF standard 1.01, aspect ratio,
density 1x1, segment length 16, baseline, precision 8, 249x256, components 3

• **Viewed Image**
└ /tmp/air_exif_thumb.jpg

• **Ran** DYLD_LIBRARY_PATH=/Users/abdullahalwakeel/anaconda3/lib /tmp/
│ scan_jpeg_coeff /tmp/air_exif_thumb.jpg
└ dyld[74173]: symbol '__ZNKSt3__119bad_expected_accessIvE4whatEv' missing
from root that overrides /usr/lib/libc++.1.dylib. Use of that symbol in /
… +9 lines
0xBAD4007.
SCAN /tmp/air_exif_thumb.jpg components=3

• **Ran** sips -s format bmp /tmp/air_exif_thumb.jpg --out /tmp/air_thumb.bmp
└ /private/tmp/air_exif_thumb.jpg
/private/tmp/air_thumb.bmp

• **Ran** /tmp/scan_lsb /tmp/air_thumb.bmp
└ SCAN /tmp/air_thumb.bmp 249x256 3ch

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from pathlib import Path
│ import re
│ … +15 lines
└ icc len 686 b'ICC_PROFILE\x00\x01\x01\x00\x00\x02\xa0lc'

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image, ImageDraw
│ import numpy as np, os
│ … +16 lines
└ Traceback (most recent call last):
File "<stdin>", line 11, in <module>
… +3 lines
'{}'".format(__name__, name))
AttributeError: module 'PIL.Image' has no attribute 'Resampling'

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image, ImageDraw
│ import numpy as np
│ … +16 lines
└ (no output)

• **Viewed Image**
└ /tmp/contact/air_all_planes.png

✔ You **approved** codex to run brew install steghide outguess **this time**

• **Waited for background terminal**
• **Waited for background terminal** · brew install steghide outguess
• **Ran** brew install steghide outguess
└ ==> Auto-updating Homebrew...
Adjust how often this is run with `$HOMEBREW_AUTO_UPDATE_SECS` or disable
… +61 lines
Error: No formulae or casks found for steghide.
==> Searching for similarly named formulae and casks...
✔ You **approved** codex to run brew install outguess **this time**
• **Ran** brew install outguess
└ Error: Cask 'outguess' has been disabled because it is no longer reliably
reachable upstream! It was disabled on 2026-02-08.
==> Fetching downloads for: outguess
• **Waited for background terminal**
✔ You **approved** codex to run /Users/abdullahalwakeel/anaconda3/bin/pip install
stegoveritas **this time**
• **Waited for background terminal**
• **Waited for background terminal** · /Users/abdullahalwakeel/anaconda3/bin/pip
install stegoveritas
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/pip install stegoveritas
└ Collecting stegoveritas
Downloading stegoveritas-1.11.tar.gz (85 kB)
… +70 lines
python-xmp-toolkit-2.1.0 stegoveritas-1.11 stegoveritas-binwalk-2.1.4
stegoveritas-pfp-0.2.3 stegoveritas-py010parser-0.1.10
— Worked for 3m 13s ——
• stegoveritas installed successfully, so I'm running it against the remaining
JPEG and searching its outputs for either direct secret{} strings or extracted
visual artifacts.
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/stegoveritas airplane.jpg -out /tmp/
│ steg_air
└ Traceback (most recent call last):
File "/Users/abdullahalwakeel/anaconda3/bin/stegoveritas", line 5, in
… +12 lines
raise ImportError('failed to find libmagic. Check your installation')
ImportError: failed to find libmagic. Check your installation
✔ You **approved** codex to run brew install libmagic **this time**
• **Ran** brew install libmagic
└ ==> Fetching downloads for: libmagic
✔ Bottle Manifest libmagic (5.47)
… +4 lines
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
✔ You **approved** codex to run DYLD_LIBRARY_PATH=/usr/local/lib:/Users/
abdullahalwakeel/anaconda3/lib /Users... **this time**
• **Waited for background terminal**
• **Waited for background terminal** · DYLD_LIBRARY_PATH=/usr/local/lib:/Users/
abdullahalwakeel/anaconda3/lib /Users/abdullahalwakeel/anaconda3/bin/
stegoveritas_install_deps
• **Ran** DYLD_LIBRARY_PATH=/usr/local/lib:/Users/abdullahalwakeel/anaconda3/lib /
│ Users/abdullahalwakeel/anaconda3/bin/stegoveritas_install_deps
└ dyld[76711]: symbol '__ZNKSt3__119bad_expected_accessIvE4whatEv' missing
from root that overrides /usr/lib/libc++.1.dylib. Use of that symbol in /
… +42 lines
to read from standard input or configure an askpass helper
sudo: a password is required
✔ You **approved** codex to run mkdir -p /Users/abdullahalwakeel/.config/binwalk/

magic /Users/abdullahalwakee... **this time**

• **Ran** mkdir -p /Users/abdullahalwakeel/.config/binwalk/magic /Users/
abdullahalwakeel/.config/binwalk/modules
└ (no output)

• **Ran** pkill -f stegoveritas
└ sysmon request failed with error: sysmond service not found
pkill: Cannot get process list

✔ You **approved** codex to run pgrep -fl stegoveritas **this time**

• **Ran** pgrep -fl stegoveritas
└ 76655 /Users/abdullahalwakeel/anaconda3/bin/python /Users/abdullahalwakeel/
anaconda3/bin/stegoveritas airplane.jpg -out /tmp/steg_air
77984 /Users/abdullahalwakeel/anaconda3/bin/python /Users/abdullahalwakeel/
anaconda3/bin/stegoveritas airplane.jpg -out /tmp/steg_air

✔ You **approved** codex to run kill 76655 77984 **this time**

• **Ran** kill 76655 77984
└ (no output)

• **Waited for background terminal** · DYLD_LIBRARY_PATH=/usr/local/lib:/Users/
abdullahalwakeel/anaconda3/lib /Users/abdullahalwakeel/anaconda3/bin/
stegoveritas airplane.jpg -out /tmp/steg_air

• **Ran** DYLD_LIBRARY_PATH=/usr/local/lib:/Users/abdullahalwakeel/anaconda3/lib /
Users/abdullahalwakeel/anaconda3/bin/stegoveritas airplane.jpg -out /tmp/
steg_air
└ dyld[77984]: symbol '__ZNKSt3__119bad_expected_accessIvE4whatEv' missing
from root that overrides /usr/lib/libc++.1.dylib. Use of that symbol in /
… +561 lines
Found something worth keeping!
BIOS (ia32) ROM Ext. (170*512) instruction 0xaaaaaaaa

• **Ran** DYLD_LIBRARY_PATH=/usr/local/lib:/Users/abdullahalwakeel/anaconda3/lib /
Users/abdullahalwakeel/anaconda3/bin/stegoveritas airplane.jpg -out /tmp/
steg_air
└ dyld[76655]: symbol '__ZNKSt3__119bad_expected_accessIvE4whatEv' missing
from root that overrides /usr/lib/libc++.1.dylib. Use of that symbol in /
… +613 lines
Found something worth keeping!
MPEG ADTS, AAC, v2, surround + side

• **Explored**
└ List steg_air

• **Ran** file /tmp/steg_air/keepers/*
└ /tmp/steg_air/keepers/1776358462.86773-5d5a0905fb8eb0364eb98d6b1b7a6166:
BIOS (ia32) ROM Ext. (72*512)
… +4 lines
/tmp/steg_air/keepers/1776358547.966788-89e5f6abcca3a6657b86de89bc35544f:
MPEG ADTS, AAC, v4 Main, 96 kHz, stereo + center

• **Explored**
└ Search secret\{[0-9A-Fa-f]+\} in steg_air

• **Ran** strings -a /tmp/steg_air/keepers/* | head -100
└ `UUh)
UUUUUUUU
… +96 lines
i(Jt5P
VUUUUUUUU

• **Ran** exiftool airplane.jpg
└ ExifTool Version Number : 13.55
File Name : airplane.jpg
… +87 lines
Thumbnail Image : (Binary data 12781 bytes, use -b option to
extract)

✗ You **canceled** the request to run brew search steg

• **Ran** brew search steg

└ (no output)

■ Conversation interrupted - tell the model what to do differently. Something went wrong? Hit `/feedback` to report the issue.

› Hint: try to use techniques other than bit-hiding.

• Thanks for the hint. I'll stop chasing raw bit streams for the remaining file and focus on non-bit-hiding paths: embedded thumbnail/metadata, visual transforms, and differences between embedded resources and the displayed image. First I'm checking whether the JPEG thumbnail differs meaningfully from the main image.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageChops, ImageOps, ImageEnhance
│ main=Image.open('airplane.jpg').convert('RGB')
│ … +9 lines
└ (1400, 1434) (249, 256) (0, 0, 249, 256)
```

✔ You **approved** codex to run pgrep -fl stegoveritas **this time**

• **Ran** pgrep -fl stegoveritas

└ (no output)

› Also, did you solve all the other images?

• Yes. The other three are solved:

mario_64.png: secret{aaff4578}

operating_system.png: secret{678800ff}

secret{_}.png: the hidden plane shows the hex text:

1f00c2f154f53e71af06dccbf7dcce6223725509ff08aab65e3ce23c3fb999cfe09f0d73a4e6451bcae8f44e28b5c39ed96c0b43786c4d93ee79e980e5c05d1d3818dc7aa6aa474013a5fdd21b7e848999

So the corresponding secret is:

secret{1f00c2f154f53e71af06dccbf7dcce6223725509ff08aab65e3ce23c3fb999cfe09f0d73a4e6451bcae8f44e28b5c39ed96c0b43786c4d93ee79e980e5c05d1d3818dc7aa6aa474013a5fdd21b7e848999}

I'm still working on airplane.jpg with non-bit-hiding techniques now.

• **Viewed Image**

└ /tmp/air_thumb_diff.png

• **Ran** exiftool -a -u -G1 airplane.jpg | rg -i "secret|comment|document|id|user|description|caption|title|subject|keyword|uuid|history|time|thumbnail"

```
└ [System]        File Modification Date/Time     : 2026:04:16 11:24:52-05:00
  [System]        File Access Date/Time           : 2026:04:16 11:49:41-05:00
  … +17 lines
  [ICC-header]    Profile ID                      : 0
  [ICC_Profile]   Profile Description             : GIMP built-in sRGB
```

• **Ran** python3 - <<'PY'

```
│ vals=['1776356692812745','20260416112447','14001434','249256','12781','304']
│ for v in vals:
│ … +2 lines
└ 1776356692812745 64f9645ab63c9
  20260416112447 126d3ee7c73f
  … +2 lines
  12781 31ed
  304 130
```

• For airplane.jpg, metadata exposes several XMP UUID-style hex values, but I don't want to treat those as the answer unless there's stronger evidence. I'm checking enhanced visual views and metadata/thumbnail artifacts first, since the hint points away from bit hiding.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageEnhance, ImageOps
│ im=Image.open('airplane.jpg').convert('RGB')
│ … +19 lines
└ (no output)
```

• **Viewed Image**
  └ /tmp/air_bright5_contrast2.png

• **Viewed Image**
  └ /tmp/air_top_enh.png

• **Viewed Image**
  └ /tmp/air_right_enh.png

› The secret{_}.png one is incorrect. Sorry, I should have better specified the format: There are exactly 8 hexadecimal numbers between the curly brackets. secret{XXXXXXXX}

• **Ran** foremost -i airplane.jpg -o /tmp/fore_air
  └ foremost: /usr/local/etc/foremost.conf: No such file or directory
  Processing: airplane.jpg
  |*|

• **Waited for background terminal**

• Understood. Then I won't wrap the long hex block as the answer. For secret{_}.png, I found a visible long hex payload in a low bit plane, but with the corrected format the actual secret must be an 8-hex value. I'm going back through that image for an exact secret{XXXXXXXX} or an 8-hex payload hidden by a non-bitstream method.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ s='1f00c2f154f53e71af06dccbf7dcce6223725509ff08aab65e3ce23c3fb999cfe09f0d73a
│ 4e6451bcae8f44e28b5c39ed96c0b43786c4d93ee79e980e5c05d1d3818dc7aa6aa474013a5f
│ … +5 lines
└ [System]       File Name                       : secret{_}.png
  [System]       File Modification Date/Time     : 2026:04:16 11:34:31-05:00
  … +39 lines
  [XMP-exif]     User Comment                    : Screenshot
  [XMP-iptcExt]  Registry ID                     :
```

• **Ran** exiftool -b -ThumbnailImage secret\{_\}.png > /tmp/secret_thumb.bin
```
│ file /tmp/secret_thumb.bin
│ rg -a -o "secret\{[0-9A-Fa-f]{8}\}" /tmp/secret_thumb.bin || true
│ … +1 lines
└ /tmp/secret_thumb.bin: JPEG image data, JFIF standard 1.01, aspect ratio,
  density 1x1, segment length 16, baseline, precision 8, 256x138, components 3
  … +18 lines
  yrMf
  s;DO
```

• **Viewed Image**
  └ /tmp/secret_thumb.bin

• **Ran** which tesseract
  └ (no output)

• **Ran** exiftool -a -u -G1 operating_system.png | rg "Document ID|Instance ID|
  │ Original|History Instance"
  └ (no output)

• **Ran** foremost -i secret\{_\}.png -o /tmp/fore_secret 2>/tmp/fore_secret.err ||
  │ true
  │ find /tmp/fore_secret -maxdepth 3 -type f | head -50
  │ … +1 lines
  └ /tmp/fore_secret/audit.txt
  /tmp/fore_secret/png/00000000.png

• **Ran** rg -a -o "[Ss][Ee][Cc][Rr][Ee][Tt]\{[^}]+\}" . /tmp/secret_thumb.bin /tmp/

```
│ air_exif_thumb.jpg /tmp/steg_air 2>/dev/null || true
└ (no output)
```

• **Viewed Image**
```
└ secret{_}.png
```

• **Ran** python3 - <<'PY'
```
│ lines=['1f90e2f154f53e71af06dccbf7dcce622','3725509ff08aab65e3ce23c3fb999cfe
│ ','09f0d73a4e6451bcae8f44e28b5c39ed','96c0b43786c4d93ee79e980e5c05d1d3','818
│ … +6 lines
└ 33 1f90e2f154f53e71af06dccbf7dcce622
  32 3725509ff08aab65e3ce23c3fb999cfe
  … +3 lines
  161
  non-hexadecimal number found in fromhex() arg at position 161
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ import binascii, zlib, gzip, bz2, lzma, math
│ s='1f90e2f154f53e71af06dccbf7dcce62'+'3725509ff08aab65e3ce23c3fb999cfe'+'09f
│ … +14 lines
└ 80
  1f90e2f154f53e71af06dccbf7dcce623725509ff08aab65e3ce23c3fb999cfe09f0d73a4e64
  … +4 lines
  \9\xed\x96\xc0\xb47\x86\xc4\xd9>\xe7\x9e\x98\x0e\\
  \x05\xd1\xd3\x81\x8d\xc7\xaaj\xa4t\x01:_\xdd!\xb7\xe8H\x99'
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ import re,zlib,gzip,bz2,lzma,base64,itertools
│ raw='1f90e2f154f53e71af06dccbf7dcce6223725509ff08aab65e3ce23c3fb999cfe09f0d7
│ … +20 lines
└ done
```

• **Ran** exiftool -s3 -DocumentID -InstanceID -OriginalDocumentID
```
│ -HistoryInstanceID airplane.jpg secret\{_\}.png
└ ======== airplane.jpg
  gimp:docid:gimp:eee6ab09-9879-4dbe-8373-df569f468687
  … +7 lines
  xmp.iid:98d200dc-4c4e-4fcb-9ce4-36ea4c537ab8
      2 image files read
```

• **Ran** python3 - <<'PY'
```
│ import zlib,hashlib
│ raw='1f90e2f154f53e71af06dccbf7dcce6223725509ff08aab65e3ce23c3fb999cfe09f0d7
│ … +10 lines
└ text crc32 6befd609 md5first8 c1edd110 sha1first8 ea7fb5f3
  drop33 crc32 fdb25a7d md5first8 ece85154 sha1first8 1181f4cd
  drop_last crc32 7e94e762 md5first8 ddb1482b sha1first8 daa7d18b
```

✗ You **canceled** the request to run gem install zsteg

• **Ran** gem install zsteg
```
└ (no output)
```

■ Conversation interrupted - tell the model what to do differently. Something went wrong? Hit `/feedback` to report the issue.

› Another hint, try going another direction! The airplane one is not far off from how you got the windows image! Try to find the text hiding in the image

• Got it. I'll focus back on visual text extraction for airplane.jpg, not metadata/tool installs. Since the Windows image revealed text by isolating a plane, I'll try other visual-domain transforms for the JPEG: channel planes, difference/high-pass, and frequency-domain views where hidden text is often placed.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image, ImageOps, ImageEnhance, ImageFilter, ImageDraw
│ import numpy as np, os
│ … +21 lines
```

└ done
✔ You **approved** codex to run pgrep -fl gem **this time**
• **Ran** pgrep -fl gem
└ 238 /System/Library/PrivateFrameworks/BackgroundTaskManagement.framework/
Resources/backgroundtaskmanagementd -daemon
… +33 lines
93757 /System/Library/PrivateFrameworks/AppleMediaServicesUI.framework/
amsengagementd
• **Viewed Image**
└ /tmp/air_visual/fft_lum.png
• **Viewed Image**
└ /tmp/air_visual/fft_r.png
• **Waited for background terminal**
• **Waited for background terminal** · /Users/abdullahalwakeel/anaconda3/bin/python
- <<'PY'

```
from PIL import Image, ImageEnhance, ImageOps
im=Image.open('airplane.jpg').convert('RGB')
# Crop suspicious diagonal/vertical purple text near upper-middle/right, rotate
and enhance channels.
boxes={
 'diag1':(560,0,850,380),
 'diag2':(620,0,820,320),
 'upperright':(520,0,1100,450),
}
for name,box in boxes.items():
 crop=im.crop(box)
 for rot in [0,90,270]:
  c=crop.rotate(rot, expand=True)
  for mode,img in [('rgb',c),('b',c.getchannel('B')),('r',c.getchannel('R')),
('g',c.getchannel('G'))]:
   if mode!='rgb': img=ImageOps.autocontrast(img).convert('RGB')
   else:
img=ImageEnhance.Contrast(ImageEnhance.Brightness(img).enhance(5)).enhance(3)
   img.resize((img.width*4,img.height*4),Image.BICUBIC).save(f'/tmp/air_visual/
{name}_rot{rot}_{mode}.png')
PY
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image, ImageEnhance, ImageOps
│ im=Image.open('airplane.jpg').convert('RGB')
│ … +20 lines
└ (no output)
• **Viewed Image**
└ /tmp/air_visual/diag1_rot0_rgb.png
• **Explored**
└ List steg_air
List steg_air
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image, ImageDraw
│ import numpy as np
│ … +17 lines
└ (no output)
• **Viewed Image**
└ /tmp/contact/air_ycbcr_planes.png
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image, ImageOps
│ im=Image.open('airplane.jpg').convert('YCbCr')
│ … +4 lines

└ (no output)
• **Viewed Image**
└ /tmp/air_visual/ycbcr_Cb.png
• **Viewed Image**
└ /tmp/air_visual/ycbcr_Cr.png
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image, ImageOps, ImageEnhance
│ im=Image.open('/tmp/air_visual/ycbcr_Cr.png')
│ … +10 lines
└ (no output)
• **Viewed Image**
└ /tmp/air_visual/cr_midright_rot90.png
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image, ImageOps, ImageDraw
│ import numpy as np, colorsys
│ … +22 lines
└ <stdin>:7: RuntimeWarning: invalid value encountered in divide
• **Waited for background terminal**
• **Viewed Image**
└ /tmp/air_visual/hue.png
• **Viewed Image**
└ /tmp/air_visual/blue_excess.png
• **Viewed Image**
└ /tmp/air_visual/red_excess.png
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image, ImageDraw
│ import numpy as np, os
│ … +26 lines
└ done
• **Viewed Image**
└ /tmp/air_visual/thresh_Ravg.png
• **Viewed Image**
└ /tmp/air_visual/thresh_Bavg.png
• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image,ImageEnhance,ImageOps,ImageFilter
│ im=Image.open('airplane.jpg').convert('RGB')
│ … +24 lines
└ (no output)
• **Waited for background terminal**
• **Viewed Image**
└ /tmp/air_visual/center_window_auto.png
• **Viewed Image**
└ /tmp/air_visual/bottom_panel_sharp.png
• **Waited for background terminal**

— Worked for 3m 35s —

• I'm going one layer deeper visually: for a JPEG, text can be hidden in the DCT coefficient planes rather than the decoded pixel planes. I'll render coefficient bit planes as images and inspect them the same way we inspected the Windows bit plane.

• **Added** dump_jpeg_coeff_planes.c (+44 -0)

```
+#include <stdio.h>
+#include <stdlib.h>
+#include <string.h>
+#include <jpeglib.h>
+
+int main(int argc, char **argv) {
+    if (argc < 3) {
```

```
 8 +        fprintf(stderr, "usage: %s input.jpg outdir\n", argv[0]);
 9 +        return 1;
10 +    }
11 +    FILE *in = fopen(argv[1], "rb");
12 +    if (!in) { perror(argv[1]); return 1; }
13 +    struct jpeg_decompress_struct cinfo;
14 +    struct jpeg_error_mgr jerr;
15 +    cinfo.err = jpeg_std_error(&jerr);
16 +    jpeg_create_decompress(&cinfo);
17 +    jpeg_stdio_src(&cinfo, in);
18 +    jpeg_read_header(&cinfo, TRUE);
19 +    jvirt_barray_ptr *coef_arrays = jpeg_read_coefficients(&cinfo);
20 +    for (int ci = 0; ci < cinfo.num_components; ci++) {
21 +        jpeg_component_info *comp = cinfo.comp_info + ci;
22 +        for (int k = 0; k < DCTSIZE2; k++) {
23 +            char path[512];
24 +            snprintf(path, sizeof(path), "%s/c%d_k%02d.pgm", argv[2], ci
   , k);
25 +            FILE *out = fopen(path, "wb");
26 +            if (!out) { perror(path); continue; }
27 +            fprintf(out, "P5\n%u %u\n255\n", comp->width_in_blocks, comp
   ->height_in_blocks);
28 +            for (JDIMENSION by = 0; by < comp->height_in_blocks; by++) {
29 +                JBLOCKARRAY row = (*cinfo.mem->access_virt_barray)
30 +                    ((j_common_ptr)&cinfo, coef_arrays[ci], by, 1, FALSE
   );
31 +                for (JDIMENSION bx = 0; bx < comp->width_in_blocks; bx++
   ) {
32 +                    int v = row[0][bx][k];
33 +                    unsigned char px = (abs(v) & 1) ? 255 : 0;
34 +                    fwrite(&px, 1, 1, out);
35 +                }
36 +            }
37 +            fclose(out);
38 +        }
39 +    }
40 +    jpeg_finish_decompress(&cinfo);
41 +    jpeg_destroy_decompress(&cinfo);
42 +    fclose(in);
43 +    return 0;
44 +}
```

- **Ran** cc -O2 dump_jpeg_coeff_planes.c -o /tmp/dump_jpeg_coeff_planes -I/Users/abdullahalwakeel/anaconda3/include -L/Users/abdullahalwakeel/anaconda3/lib -ljpeg
  … +3 lines
  └ dyld[79814]: symbol '__ZNKSt3__119bad_expected_accessIvE4whatEv' missing from root that overrides /usr/lib/libc++.1.dylib. Use of that symbol in /
  … +8 lines
  Library/PrivateFrameworks/caulk.framework/Versions/A/caulk is being set to 0xBAD4007.
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  from PIL import Image, ImageDraw, ImageOps
  import glob, os, math
  … +13 lines
  └ (no output)
- **Waited for background terminal**
- **Viewed Image**

└ /tmp/air_coeff/contact_c0.png

• **Viewed Image**

└ /tmp/air_coeff/contact_c1.png

• **Edited** dump_jpeg_coeff_planes.c (+28 -0)

```
37          fclose(out);
38 +        snprintf(path, sizeof(path), "%s/c%d_k%02d_sign.pgm", argv[2
    ], ci, k);
39 +        out = fopen(path, "wb");
40 +        if (!out) { perror(path); continue; }
41 +        fprintf(out, "P5\n%u %u\n255\n", comp->width_in_blocks, comp
    ->height_in_blocks);
42 +        for (JDIMENSION by = 0; by < comp->height_in_blocks; by++) {
43 +            JBLOCKARRAY row = (*cinfo.mem->access_virt_barray)
44 +                ((j_common_ptr)&cinfo, coef_arrays[ci], by, 1, FALSE
    );
45 +            for (JDIMENSION bx = 0; bx < comp->width_in_blocks; bx++
    ) {
46 +                int v = row[0][bx][k];
47 +                unsigned char px = v < 0 ? 255 : 0;
48 +                fwrite(&px, 1, 1, out);
49 +            }
50 +        }
51 +        fclose(out);
52 +        snprintf(path, sizeof(path), "%s/c%d_k%02d_nz.pgm", argv[2],
     ci, k);
53 +        out = fopen(path, "wb");
54 +        if (!out) { perror(path); continue; }
55 +        fprintf(out, "P5\n%u %u\n255\n", comp->width_in_blocks, comp
    ->height_in_blocks);
56 +        for (JDIMENSION by = 0; by < comp->height_in_blocks; by++) {
57 +            JBLOCKARRAY row = (*cinfo.mem->access_virt_barray)
58 +                ((j_common_ptr)&cinfo, coef_arrays[ci], by, 1, FALSE
    );
59 +            for (JDIMENSION bx = 0; bx < comp->width_in_blocks; bx++
    ) {
60 +                int v = row[0][bx][k];
61 +                unsigned char px = v != 0 ? 255 : 0;
62 +                fwrite(&px, 1, 1, out);
63 +            }
64 +        }
65 +        fclose(out);
66      }
```

• **Ran** cc -O2 dump_jpeg_coeff_planes.c -o /tmp/dump_jpeg_coeff_planes -I/Users/
abdullahalwakeel/anaconda3/include -L/Users/abdullahalwakeel/anaconda3/lib
-ljpeg
… +22 lines
└ dyld[80036]: symbol '__ZNKSt3__119bad_expected_accessIvE4whatEv' missing
from root that overrides /usr/lib/libc++.1.dylib. Use of that symbol in /
… +8 lines
Library/PrivateFrameworks/caulk.framework/Versions/A/caulk is being set to
0xBAD4007.

• **Waited for background terminal**

› Status update, were you able to find either one of the remaining secrets?

• **Waited for background terminal**

— Worked for 2m 14s ——

• Status: not confirmed yet.

What is solid:

- mario_64.png: secret{aaff4578}
- operating_system.png: secret{678800ff}

What changed:

- The earlier secret{_}.png answer was wrong under the corrected format. The image visibly contains a long hex string, but I have not confirmed which 8-hex value is the intended secret. I'm treating it as unsolved for now.

Current work on airplane.jpg:

- Raw strings/appended-data/metadata did not reveal a direct secret{XXXXXXXX}.
- Pixel RGB/YCbCr bit planes did not show the text.
- FFT/frequency views did not show readable text.
- I'm now inspecting JPEG DCT coefficient visual planes, which is closer to the Windows-image method but in JPEG space. So far no confirmed readout, but this is the most promising path after your hint.

• **Viewed Image**
└ /tmp/air_coeff2/contact_c0_sign.png

• **Viewed Image**
└ /tmp/air_coeff2/contact_c1_sign.png

• The DCT coefficient visual planes are not showing readable text. I'm going to test a simpler interpretation of "another direction": column-major/reversed traversal of the decoded image planes, in case the text is encoded as a stream but laid out in a different scan direction.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image
│ import numpy as np,re,itertools
│ … +18 lines
└ (no output)
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image, ImageChops, ImageEnhance, ImageOps
│ im=Image.open('airplane.jpg').convert('RGB')
│ … +9 lines
└ (no output)
```

• **Waited for background terminal**

• **Viewed Image**
└ /tmp/air_visual/ela_auto_q90.png

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image, ImageOps
│ im=Image.open('/tmp/air_visual/ela_auto_q90.png')
│ … +7 lines
└ (no output)
```

• **Viewed Image**
└ /tmp/air_visual/ela_diag_rot90.png

• **Waited for background terminal**

— Worked for 2m 19s ——

• One more visual route I'm checking: histogram-style hiding. It is less common, but it matches "text hiding in the image" without raw bit extraction.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image, ImageDraw, ImageOps
│ import numpy as np, os
│ … +22 lines
└ (no output)
```

• **Viewed Image**
└ /tmp/air_visual/hist2_RB.png

✔ You **approved** codex to run pgrep -fl "python|dump_jpeg|scan_jpeg|gem|stegoveritas" **this time**

• **Ran** pgrep -fl "python|dump_jpeg|scan_jpeg|gem|stegoveritas"
```
└ 238 /System/Library/PrivateFrameworks/BackgroundTaskManagement.framework/
  Resources/backgroundtaskmanagementd -daemon
```

… +43 lines
93757 /System/Library/PrivateFrameworks/AppleMediaServicesUI.framework/amsengagementd

---

• Current confirmed secrets:
- mario_64.png: secret{aaff4578}
- operating_system.png: secret{678800ff}

Still not confirmed:
- secret{_}.png: my earlier long-hex answer is invalid under the corrected secret{XXXXXXXX} format. I found the visible long hex payload, but not the intended 8-hex secret yet.
- airplane.jpg: not solved yet. I checked raw strings, appended data, EXIF/XMP, embedded thumbnail, RGB/YCbCr planes, traversal directions, FFT/ELA, and JPEG DCT coefficient visual planes. None has yielded a confirmed secret{XXXXXXXX} so far.

No extraction/analyzer processes are still running.

› Ok, I'll give you another hint: The secret{_} one is readable instantly once you know which letters to choose, and which to ignore. Maybe looking more closely at the image (zoomed in?) can help you figure it out. The airplane one is readable instantly once you do the right visual transformations of the image, and there are two done for the airplane one.

• I'll focus on the two visual puzzles directly now. For secret{_}.png, I'm going to zoom into the text/dot pattern and extract the character positions; for airplane.jpg, I'll try paired visual transforms, since you said two transformations are needed.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image
│ im=Image.open('secret{_}.png').convert('RGB')
│ … +8 lines
└ (1272, 686) (1100, 370)
```

• **Viewed Image**
└ /tmp/secret_zoom3.png

• The zoom confirms the visible text is a long hex-looking string with the red spellcheck underline under essentially every character. I'm going to treat the long string as a decoy/carrier and test selection rules over its characters, plus inspect whether the underline itself marks any characters differently.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ s='1f90e2f154f53e71af06dccbf7dcce6223725509ff08aab65e3ce23c3fb999cfe09f0d73a
│ 4e6451bcae8f44e28b5c39ed96c0b43786c4d93ee79e980e5c05d1d3818dc7aa6aa474013a5f
│ … +13 lines
└ len 161
  letters feffeafdccbfdcceffaabececfbcfefdaebcaefebcedcbcdeeeecdddcaaaaafddbe
  … +230 lines
  n 20 off 19 first8 699bc0a9
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image
│ import numpy as np
│ … +39 lines
└ red pixels 13805 bbox 89 147 1149 410
  components 411
  … +28 lines
    (35, 128, 405, 134, 410, 131.05714285714285, 407.65714285714284)
    (33, 141, 405, 146, 410, 143.42424242424244, 407.57575757575756)
```

- The red underline is just spellcheck-style dotted underline, not one marker per character. I'm checking the glyphs themselves now: if the intended 8 characters are "instantly readable" when zoomed, they may be a different shade/color/weight than the decoy characters.
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image, ImageDraw
  │ import numpy as np, os
  │ … +49 lines
  └ top colored
  (3, 29, '1', (1071, 306, 1111, 381), 625, 11, (5.3936, 6.2976, 7.4704))
  … +56 lines
  (4, 29, '8', (1026, 387, 1065, 462), 148, 0, (6.162162162162162, 6.70945945945946, 7.358108108108108))
- **Viewed Image**
  └ /tmp/secret_glyph_outliers.png
- **Ran** python3 - <<'PY'
  │ s='1f90e2f154f53e71af06dccbf7dcce6223725509ff08aab65e3ce23c3fb999cfe09f0d73a
  │ 4e6451bcae8f44e28b5c39ed96c0b43786c4d93ee79e980e5c05d1d3818dc7aa6aa474013a5f
  │ … +12 lines
  └ feffeafdccbfdcceffaabececfbcfefdaebcaefebcedcbcdeeeecdddcaaaaafddbe
  deadbeef -1 -1
  … +7 lines
  every8 letters chunks ['feffeafd', 'ccbfdcce', 'ffaabece', 'cfbcfefd', 'aebcaefe', 'bcedcbcd', 'eeeecddd', 'caaaaafd', 'dbe']
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image
  │ im=Image.open('secret{_}.png').convert('RGB')
  │ … +8 lines
  └ (no output)
- **Viewed Image**
  └ /tmp/secret_line1_x4.png
- **Viewed Image**
  └ /tmp/secret_line5_x4.png

— Worked for 1m 40s —

- For airplane.jpg, the most suspicious visible artifact is a faint diagonal/purple string in the upper middle-right. I'm isolating that area with channel extraction plus rotation/contrast, since that matches "two visual transformations" better than more stego tooling.
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image, ImageOps, ImageEnhance, ImageFilter
  │ import numpy as np, os
  │ … +31 lines
  └ (no output)
- **Viewed Image**
  └ /tmp/air_diag/orig_rot45.png
- **Viewed Image**
  └ /tmp/air_diag/Bexcess_rot45.png
- **Waited for background terminal**

— Worked for 1m 03s —

- I'm taking the "direction" hint literally too: making rotated/flipped + inverted/contrast views of the whole airplane image and crops. If the two transforms are things like invert plus rotate/flip, the text should pop out.
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image, ImageOps, ImageEnhance, ImageDraw
  │ im=Image.open('airplane.jpg').convert('RGB')

│ … +30 lines
└ (no output)

• **Viewed Image**
└ /tmp/air_visual/direction_contact.png

• **Waited for background terminal**

— Worked for 2m 32s —

• I don't see the airplane text in the direction/inversion contact sheet. I'm going to generate a StegSolve-style contact sheet of color/bit/threshold combinations at larger scale around the full image, because the transform may be a specific channel plus a display operation.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image, ImageDraw, ImageOps
│ import numpy as np
│ … +22 lines
└ (no output)

• **Viewed Image**
└ /tmp/air_visual/stegsolve_large.png

⚠ Heads up, you have less than 25% of your 5h limit left. Run /status for a breakdown.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image
│ import numpy as np
│ … +14 lines
└ alpha unique sample/count [255] 1 minmax 255 255
dark count 65821 alpha dark uniques [255]
alpha!=255 0

• **Ran** python3 - <<'PY'
│ letters='feffeafdccbfdcceffaabececfbcfefdaebcaefebcedcbcdeeeecdddcaaaaafddbe
│ '
│ … +5 lines
└ 2 0 ffefcbdcfabccbffabaf
2 1 efadcfcefaeefcedecee
… +48 lines
9 8 cfbccdf

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image
│ import numpy as np
│ … +30 lines
└ count distribution Counter({102: 12, 96: 10, 92: 9, 78: 8, 82: 7, 77: 7, 81: 7, 80: 7, 90: 7, 86: 6, 91: 5, 98: 5, 72: 5, 93: 5, 100: 5, 89: 4, 97: 4,
… +55 lines
'5'), (102, 1, 0, '3'), (101, 3, 15, 'e'), (101, 3, 11, '4'), (101, 3, 7, '7')] 7b48ca80

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image,ImageDraw
│ im=Image.open('secret{_}.png').convert('RGB')
│ … +16 lines
└ selected 10974289

• **Viewed Image**
└ /tmp/secret_mark_cnt78.png

• **Waited for background terminal**

— Worked for 4m 13s —

• I noticed an important discrepancy: the normal image text and the low-plane rendered text are not identical. That likely matches your hint: choose the characters that differ, ignore the rest. I'm comparing those two visible strings directly now.

• **Ran** sips -s format png /tmp/planes/secret_p0_c0.pgm --out /tmp/

│ secret_plane_c0.png >/dev/null
│ /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ … +5 lines
└ (no output)

• **Viewed Image**
└ /tmp/secret_plane_c0_zoom.png

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image
│ import numpy as np
│ … +37 lines
└ tol 0.5 n 16 2c630c3f744ee5a5 [(0, 5, '2', 0.4090909090909065), (0, 21, 'c', 0.060606060606005485), (0, 30, '6', 0.3484848484849863), (1, 0, '3',
… +72 lines
2.6212121212121247), (0, 28, 'c', 1.0454545454545041), (0, 30, '6', 0.3484848484849863)]

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
│ from PIL import Image, ImageDraw, ImageOps, ImageEnhance
│ import numpy as np
│ … +24 lines
└ (no output)

• **Viewed Image**
└ /tmp/air_visual/gamma_contact.png

• **Waited for background terminal**

— Worked for 2m 50s —

• I have a plausible candidate for secret{_}.png: selecting the characters that align with the repeated underline-dot pattern gives 10974289. I'm not fully satisfied yet, but it is the first 8-character result that comes from an actual visual selection rule rather than a guess.
For airplane, I'm going to try OCR on generated visual variants to avoid relying only on my visual inspection.

✔ You **approved** codex to run brew install tesseract **this time**

• **Waited for background terminal** · brew install tesseract

• **Ran** brew install tesseract
└ ✔ JSON API cask.jws.json
✔ JSON API formula.jws.json
… +194 lines
If you need any other supported languages, run `brew install tesseract-lang`.

• **Waited for background terminal**

• **Waited for background terminal** · for f in /tmp/air_visual/*.png /tmp/air_diag/*.png /tmp/air_coeff2/*.png; do base=$(basename "$f"); out=$(tesseract "$f" stdout --psm 6 2>/dev/null | tr -d '\n'); if echo "$out" | rg -qi 'secret|[0-9a-f]{8}'; then echo "=== $base ==="; echo "$out" | head -c 300; echo; fi; done

• **Ran** for f in /tmp/air_visual/*.png /tmp/air_diag/*.png /tmp/air_coeff2/*.png;
│ do base=$(basename "$f"); out=$(tesseract "$f" stdout --psm 6 2>/dev/null
│ | tr -d '\n'); if echo "$out" | rg -qi 'secret|[0-9a-f]{8}'; then echo
│ … +1 lines
└ (no output)

• **Waited for background terminal**

— Worked for 5m 12s —

• Current state:
Confirmed:
- mario_64.png: secret{aaff4578}
- operating_system.png: secret{678800ff}
Likely but not fully confirmed:
- secret{_}.png: secret{10974289}
Why: after zooming into the visible text and treating the red dotted underline

as the selection/ignore cue, the characters that align consistently under the marker pattern produce 10974289. I'm not calling this 100% confirmed because the visual rule is subtle and I'd want one more validating cue.

Still unsolved:

- airplane.jpg

For the airplane image I tried the likely "two visual transformations" paths: channel isolation, inversion, gamma/levels, rotations/flips, RGB/YCbCr planes, ELA, FFT, and DCT coefficient visual planes. I haven't yet found a readable secret{XXXXXXXX}.

› Final hints: Look at the background colors in secret{_}.png around the letters. The red dotted underline is a red herring (!). You were right about something in the middle-top-right in airplane photo, did you turn up the contrast and apply a warp transform to straighten out the text?

• I'll ignore the underline now and sample the background immediately around each glyph in secret{_}.png. For airplane.jpg, I'll go back to the faint middle-top-right text, boost contrast, then deskew/warp that local region instead of just rotating it.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageDraw
│ import numpy as np, statistics
│ … +54 lines
└ global median [212.69290505 231.17779473 254.0465035 ]
  top dist
  … +29 lines
  1 11 5 mean (222.03, 236.63, 254.43) dist 10.82 bg 2606
  top8 string f201fa49
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageDraw
│ import numpy as np
│ … +51 lines
└ top rowdiff
  1 32 2 diff 2.25 signed (-2.04, -0.8, 0.51) mean (227.6, 239.99, 255.0)
  … +38 lines
  2 23 3 diff 0.75 signed (-0.35, -0.46, -0.47) mean (212.35, 229.91, 252.57)
  top8 2ef10bbe
```

— Worked for 1m 41s ——————————————————————————————————————

• The row-normalized numeric outlier method is noisy, so I'm switching to a direct visual enhancement of the background only: remove/dim the black glyphs and red underline, then amplify tiny background color differences. That should show which letters have distinct background color.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageFilter, ImageEnhance, ImageOps
│ import numpy as np
│ … +37 lines
└ (no output)
```

⚠ Heads up, you have less than 10% of your 5h limit left. Run /status for a breakdown.

• **Viewed Image**

└ /tmp/secret_bg_res_amp16.png

• **Viewed Image**

└ /tmp/secret_bg_BminusR_amp32.png

• **Waited for background terminal**

— Worked for 2m 19s ——————————————————————————————————————

• Context compacted

⚠ Heads up, you have less than 5% of your 5h limit left. Run /status for a breakdown.

• I'm going back into the two unsolved images with the new hints: for

secret{_}.png I'm going to treat background color as the selector, and for the airplane I'll focus on the confirmed middle-top-right artifact with contrast plus geometric warping.

- **Explored**
  └ Search *secret* in tmp
  List ls -la

- The previous generated scratch files are gone from /tmp, so I'm recreating the focused diagnostics rather than relying on stale intermediates.
- **Ran** identify -verbose airplane.jpg | head -60
  └ /bin/bash: identify: command not found
- **Ran** identify -verbose 'secret{_}.png' | head -60
  └ /bin/bash: identify: command not found
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image
  │ for f in
  │ … +4 lines
  └ secret{_}.png (1272, 686) RGBA
  airplane.jpg (1400, 1434) RGB
  operating_system.png (1152, 864) RGBA
  mario_64.png (640, 480) RGBA
- **Waited for background terminal** · /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
from PIL import Image, ImageEnhance, ImageFilter, ImageOps, ImageDraw
import numpy as np, os
im=Image.open('secret{_}.png').convert('RGB')
arr=np.array(im).astype(np.int16)
# mask out dark text and red underline-ish pixels
R,G,B=arr[:,:,0],arr[:,:,1],arr[:,:,2]
mask_text=(R<90)&(G<90)&(B<90)
mask_red=(R>130)&(G<100)&(B<100)&((R-G)>50)
mask=mask_text|mask_red
# estimate smooth background by median blur on original (helps remove chars)
# PIL median filter per channel
bg=im.filter(ImageFilter.MedianFilter(21))
bgarr=np.array(bg).astype(np.int16)
# residual from local row/column smooth? amplify saturation and contrast
for name, img in [('bg_med21', bg), ('sat8', ImageEnhance.Color.enhance(bg,8) if
False else bg)]:
    pass
# make amplified deviation from grayscale/local average
# subtract heavily blurred bg to remove global paper gradient, amplify chroma
blur=bg.filter(ImageFilter.GaussianBlur(35))
ba=np.array(bg).astype(np.int16); bl=np.array(blur).astype(np.int16)
res=(ba-bl)*12+128
res=np.clip(res,0,255).astype(np.uint8)
Image.fromarray(res).save('/tmp/secret_bg_chroma_res12.png')
# Saturation only (HSV S amplified)
import colorsys
rgb=np.array(bg).astype(np.float32)/255.0
mx=rgb.max(2); mn=rgb.min(2); sat=np.where(mx==0,0,(mx-mn)/mx)
hue=np.zeros(mx.shape)
# create false color from signed channels against gray
gray=ba.mean(2,keepdims=True)
ch=(ba-gray)*20+128
Image.fromarray(np.clip(ch,0,255).astype(np.uint8)).save('/tmp/
```

```
secret_chroma20.png')
# enlarge main text area
crop=Image.fromarray(np.clip(ch,0,255).astype(np.uint8)).crop((60,40,1220,500)).
resize((2320,920), Image.Resampling.NEAREST)
crop.save('/tmp/secret_chroma20_zoom.png')
# original zoom too
im.crop((60,40,1220,500)).resize((2320,920), Image.Resampling.NEAREST).save('/
tmp/secret_orig_zoom.png')
print('/tmp/secret_chroma20_zoom.png /tmp/secret_orig_zoom.png')
PY
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageEnhance, ImageFilter, ImageOps, ImageDraw
│ import numpy as np, os
│ … +42 lines
└ Traceback (most recent call last):
    File "<stdin>", line 34, in <module>
  … +3 lines
  '{}'".format(__name__, name))
  AttributeError: module 'PIL.Image' has no attribute 'Resampling'
```

• **Waited for background terminal** · /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
from PIL import Image, ImageFilter
import numpy as np
im=Image.open('secret{_}.png').convert('RGB')
bg=im.filter(ImageFilter.MedianFilter(21))
blur=bg.filter(ImageFilter.GaussianBlur(35))
ba=np.array(bg).astype(np.int16); bl=np.array(blur).astype(np.int16)
res=(ba-bl)*12+128
Image.fromarray(np.clip(res,0,255).astype(np.uint8)).save('/tmp/
secret_bg_res12.png')
gray=ba.mean(2,keepdims=True)
ch=(ba-gray)*20+128
ch_img=Image.fromarray(np.clip(ch,0,255).astype(np.uint8))
ch_img.save('/tmp/secret_chroma20.png')
ch_img.crop((60,40,1220,500)).resize((2320,920), Image.NEAREST).save('/tmp/
secret_chroma20_zoom.png')
im.crop((60,40,1220,500)).resize((2320,920), Image.NEAREST).save('/tmp/
secret_orig_zoom.png')
print('/tmp/secret_chroma20_zoom.png')
PY
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageFilter
│ import numpy as np
│ … +17 lines
└ /tmp/secret_chroma20_zoom.png
```

• **Viewed Image**

```
└ /tmp/secret_chroma20_zoom.png
```

• **Viewed Image**

```
└ /tmp/secret_orig_zoom.png
```

• On secret{_}.png, the clue is not embedded text or the underline; I'm now measuring the local background color around each displayed hex character and looking for the eight characters whose cells differ from the surrounding row.

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageDraw
│ import numpy as np, cv2, math, statistics
```

```
│ … +46 lines
└ rows [32, 32, 32, 32, 32]
 0 32 len 33
 … +2 lines
 3 32 len 32
 4 32 len 32
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageDraw
│ import numpy as np, cv2, statistics, math
│ … +83 lines
└
 TOP score
 … +62 lines
 1:18 0 med=[213. 232. 255.] d=[0. 0. 0.] score=0.00 warm=0.0 green=0.0
 1:19 6 med=[213. 232. 255.] d=[0. 0. 0.] score=0.00 warm=0.0 green=0.0
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image
│ import numpy as np
│ … +10 lines
└ (686, 1272, 4) RGBA
 R 247 [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19] [229 230
 … +8 lines
 A 1 [255] [255]
 alpha non255 0 min 255 max 255
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageEnhance
│ im=Image.open('secret{_}.png').convert('RGB')
│ … +8 lines
└ done
```

• **Viewed Image**

```
└ /tmp/secret_sat16_zoom.png
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageDraw
│ import numpy as np, cv2, collections, math
│ … +73 lines
└ row 1 mode [213 232 255] count [((213, 232, 255), 41206), ((255, 255, 255),
 672), ((137, 180, 251), 245), ((212, 231, 254), 69), ((185, 202, 222), 64)]
 … +50 lines
 1:03 0 nonbase=24 score=221.7 uniq=12
 top8 string order: 96e66666
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageDraw
│ import numpy as np, cv2
│ … +48 lines
└
 TOP white
 … +65 lines
 1:15 1 light=0 base=965 sel=0
 top8 order 024fedb7
```

• **Viewed Image**

```
└ /tmp/secret_nonbase_top.png
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageDraw
│ import numpy as np, cv2
│ … +43 lines
└ [{'ri': 0, 'j': 3, 'ch': '0', 'white': 28, 'box': (168, 93, 202, 150)},
 {'ri': 0, 'j': 5, 'ch': '2', 'white': 36, 'box': (234, 93, 266, 150)},
```

… +5 lines
{'ri': 4, 'j': 25, 'ch': '7', 'white': 79, 'box': (868, 351, 897, 408)}]
secret{024fedb7}

- **Waited for background terminal**

— Worked for 2m 38s —

- For secret{_}.png, that hint paid off: there are exactly eight characters with pure white background pixels in their character cells, in reading order giving secret{024fedb7}. I'm switching fully to the airplane now.
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image, ImageEnhance, ImageOps
  │ im=Image.open('airplane.jpg').convert('RGB')
  │ … +29 lines
  └ generated
- **Viewed Image**
  └ /tmp/air_broad_contrast.png
- **Viewed Image**
  └ /tmp/air_rightpanel_contrast.png
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image, ImageEnhance, ImageOps
  │ import numpy as np, cv2
  │ … +35 lines
  └ done
- **Waited for background terminal**
- **Viewed Image**
  └ /tmp/air_strip1_color_x6.png
- **Viewed Image**
  └ /tmp/air_strip1_BminusR_x6.png
- **Viewed Image**
  └ /tmp/air_strip1_cyan_x6.png
- **Viewed Image**
  └ /tmp/air_strip2_color_x6.png
- **Viewed Image**
  └ /tmp/air_strip3_color_x6.png
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image
  │ for src in ['/tmp/air_strip1_color_x6.png','/tmp/
  │ … +6 lines
  └ done
- **Waited for background terminal**
- **Viewed Image**
  └ /tmp/air_strip1_color_x6_rot90.png
- **Viewed Image**
  └ /tmp/air_strip1_color_x6_rot270.png
- **Waited for background terminal**

— Worked for 2m 08s —

- For the airplane, the confirmed artifact is a faint blue/cyan string running almost vertically on the right side of the cockpit crop. I can see the character shapes after contrast; now I'm warping that strip into a horizontal reading angle.
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image, ImageEnhance, ImageOps, ImageDraw
  │ import numpy as np, cv2, math, os
  │ … +48 lines
  └ /tmp/air_warp_contact.png 54
- **Viewed Image**
  └ /tmp/air_warp_contact.png
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
  │ from PIL import Image, ImageEnhance, ImageOps
  │ import numpy as np
  │ … +22 lines
  └ done
• Waited for background terminal
• Viewed Image
  └ /tmp/air_textA_color_x7.png
• Ran /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image
  │ for src in ['/tmp/air_textA_color_x7.png','/tmp/air_textB_color_x7.png','/
  │ … +5 lines
  └ (no output)
• Viewed Image
  └ /tmp/air_textA_color_x7_rot270.png
• Viewed Image
  └ /tmp/air_textA_color_x7_rot90.png
• Waited for background terminal
• Ran pgrep -fl python
  └ 2299 /Users/abdullahalwakeel/.vscode/extensions/
    ms-python.python-2026.4.0-darwin-x64/python-env-tools/bin/pet server
    … +62 lines
    PY
    91116 /Users/abdullahalwakeel/anaconda3/bin/python -
✔ You approved codex to run kill 91115 91116 this time
• Ran kill 91115 91116
  └ (no output)
• Waited for background terminal · /Users/abdullahalwakeel/anaconda3/bin/python
- <<'PY'
from PIL import Image, ImageEnhance, ImageOps
import numpy as np, cv2, math
im=Image.open('airplane.jpg').convert('RGB')
# full coords centerline along hidden text, manually approximated from contrast
crop
points=np.array([
 [965,125], [965,155], [970,185], [982,220], [992,255], [1011,300], [1026,345]
], dtype=float)
# resample centerline
segments=[]; total=0
for a,b in zip(points[:-1],points[1:]):
 v=b-a; L=np.linalg.norm(v); segments.append((a,b,L)); total+=L
W=int(total*5); H=180; half_width=28
# output x maps along centerline, y maps perpendicular
out=np.zeros((H,W,3),dtype=np.uint8)
for xi in range(W):
 s=xi/(W-1)*total
 acc=0
 for a,b,L in segments:
  if acc+L>=s:
   t=(s-acc)/L if L else 0; p=a+(b-a)*t; tang=(b-a)/L; break
  acc+=L
 # perpendicular
 perp=np.array([-tang[1], tang[0]])
 for yi in range(H):
  off=(yi/(H-1)-0.5)*2*half_width
  src=p+perp*off
  x,y=src
  if 0<=x<im.width-1 and 0<=y<im.height-1:
```

```
  out[yi,xi]=cv2.getRectSubPix(np.array(im), (1,1), (float(x),float(y)))[0,0]
img=Image.fromarray(out)
# Try both orientations/enhancements
for flipname,base in [('normal',img),
('flip',img.transpose(Image.FLIP_LEFT_RIGHT))]:
 enh=ImageOps.autocontrast(base,cutoff=1)
 enh=ImageEnhance.Contrast(enh).enhance(6)
 enh=ImageEnhance.Color(enh).enhance(4)
 enh=ImageEnhance.Sharpness(enh).enhance(2)
 enh.resize((W*2,H*2),Image.BICUBIC).save(f'/tmp/air_curve_{flipname}
_color.png')
 # also threshold blue channel image
 arr=np.array(base).astype('int16'); R,G,B=arr[:,:,0],arr[:,:,1],arr[:,:,2]
 m=(B+G-2*R).astype('float32')
 lo,hi=np.percentile(m,[2,99.7]); gray=np.clip((m-lo)/(hi-
lo)*255,0,255).astype('uint8')
 gi=Image.fromarray(gray); gi=ImageOps.autocontrast(gi);
gi=ImageEnhance.Contrast(gi).enhance(4)
 gi.resize((W*2,H*2),Image.BICUBIC).save(f'/tmp/air_curve_{flipname}_cyan.png')
print(W,H)
PY
```

- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image, ImageEnhance, ImageOps
  │ import numpy as np, cv2, math
  │ … +51 lines
  └ (no output)
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
  │ from PIL import Image, ImageEnhance, ImageOps, ImageDraw
  │ import numpy as np, cv2, math
  │ … +60 lines
  └ /tmp/air_curve_contact.png
- **Viewed Image**
  └ /tmp/air_curve_contact.png
- **Viewed Image**
  └ /tmp/air_curve_p2_h28_n.png
- **Viewed Image**
  └ /tmp/air_curve_p2_h28_f.png
- **Ran** tesseract /tmp/air_curve_p2_h28_n.png stdout --psm 7 -c
  │ tessedit_char_whitelist='secret{}0123456789abcdef'
  └ Error in fopenReadStream: failed to open locally with tail
    air_curve_p2_h28_n.png for filename /tmp/air_curve_p2_h28_n.png
    … +5 lines
    Image file PNG cannot be read!
    Error during processing.
- **Explored**
  └ List air_curve_*
- **Ran** tesseract /tmp/air_curve_p2_h28_n.png stdout --psm 7
  └ Error in fopenReadStream: failed to open locally with tail
    air_curve_p2_h28_n.png for filename /tmp/air_curve_p2_h28_n.png
    … +5 lines
    Image file PNG cannot be read!
    Error during processing.
- **Ran** cp /tmp/air_curve_p2_h28_n.png ./air_curve_p2_h28_n.png
  └ (no output)
- **Ran** tesseract air_curve_p2_h28_n.png stdout --psm 7
  └ (no output)
- **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageOps, ImageEnhance, ImageFilter
│ import numpy as np, cv2
│ … +51 lines
└ 1796 180
```

• **Waited for background terminal**

• **Viewed Image**
```
└ /tmp/air_clean_color.png
```

• **Viewed Image**
```
└ /tmp/air_clean_white_bw_inv.png
```

• **Viewed Image**
```
└ /tmp/air_curve_p3_h28_n.png
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image, ImageEnhance, ImageOps
│ for src in ['/tmp/air_clean_color.png','/tmp/air_curve_p3_h28_n.png','/tmp/
│ … +11 lines
└ (no output)
```

• **Waited for background terminal**

• **Viewed Image**
```
└ /tmp/crop_air_clean_color_band.png
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image, ImageEnhance, ImageOps, ImageDraw
│ import numpy as np, cv2, math
│ … +36 lines
└ done
```

• **Waited for background terminal**

• **Viewed Image**
```
└ /tmp/air_shift_contact.png
```

• **Viewed Image**
```
└ /tmp/air_shift_0.png
```

• **Viewed Image**
```
└ /tmp/air_shift_-10.png
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image
│ for s in ['-10','0','10']:
│ … +3 lines
└ (no output)
```

• **Viewed Image**
```
└ /tmp/air_shift_0_flip.png
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'
```
│ from PIL import Image, ImageDraw
│ img=Image.open('/tmp/air_shift_0.png')
│ … +14 lines
└ (3142, 340)
```

• **Waited for background terminal**

• **Viewed Image**
```
└ /tmp/air_shift_0_grid.png
```

• **Viewed Image**
```
└ /tmp/air_charcrop_0.png
```

• **Viewed Image**
```
└ /tmp/air_charcrop_1.png
```

• **Viewed Image**
```
└ /tmp/air_charcrop_2.png
```

• **Viewed Image**
```
└ /tmp/air_charcrop_3.png
```

• **Viewed Image**
```
└ /tmp/air_charcrop_4.png
```

• **Ran** /Users/abdullahalwakeel/anaconda3/bin/python - <<'PY'

```
│ from PIL import Image, ImageOps
│ import numpy as np, cv2
│ … +19 lines
└ done
```

• **Viewed Image**
└ /tmp/air_mask_full.png

• **Ran** cp /tmp/air_mask_full.png ./air_mask_full.png && tesseract

```
│ air_mask_full.png stdout --psm 7 -c tessedit_char_whitelist=secret{}
│ 0123456789abcdef
└ (no output)
```

• **Ran** pgrep -fl '/Users/abdullahalwakeel/anaconda3/bin/python -'

```
└ sysmon request failed with error: sysmond service not found
  pgrep: Cannot get process list
```

• **Waited for background terminal**

— Worked for 13m 55s ——

• I found the remaining two.

All four secrets:

- secret{aaff4578} from mario_64.png
- secret{678800ff} from operating_system.png
- secret{024fedb7} from secret{_}.png
- secret{0bbcf00b} from airplane.jpg

For the last two: secret{_}.png used the pure-white background pixels around exactly eight hex characters, read in order. The airplane one came from boosting contrast on the faint blue/cyan text in the middle-top-right cockpit area, then straightening the curved/warped strip into a horizontal line.